Глубокое обучение с подкреплением

Модуль 8: оптимизация политики PPO

Курс Hugging Face Deep RL

Полная учебная версия на основе MDX

Источник: курс Hugging Face Deep RL (оригинал на английском языке).

Содержание

1 Введение **2**

2 Интуиция, лежащая в основе PPO **3**

3 Знакомство с суррогатной целевой функцией с ограничением **4**
- 3.1 Повторение: целевая функция политики . . . 4
- 3.2 Функция отношения вероятностей . . . 5
- 3.3 Часть суррогатной целевой функции без ограничения . . . 6
- 3.4 Часть суррогатной целевой функции с ограничением . . . 7

4 Визуализация суррогатной целевой функции с ограничением **8**
- 4.1 Случаи 1 и 2: отношение находится внутри интервала . . . 9
- 4.2 Случаи 3 и 4: отношение ниже интервала . . . 10
- 4.3 Случаи 5 и 6: отношение выше интервала . . . 10

5 Практическая часть **12**
- 5.1 Модуль 8: PPO на PyTorch . . . 13
- 5.2 Цели практического ноутбука . . . 13
- 5.3 Предварительные требования . . . 13
- 5.4 Настройка GPU . . . 13
- 5.5 Создание виртуального экрана . . . 14
- 5.6 Установка зависимостей . . . 15
- 5.7 Реализуем PPO с нуля по руководству Косты Хуанга . . . 15
- 5.8 Добавление интеграции с Hugging Face . . . 15
- 5.9 Запуск обучения . . . 30
- 5.10 Дополнительные задания . . . 31
- 5.11 Продолжайте учиться и совершенствоваться . . . 31

6 Заключение **31**
- 6.1 Продолжайте учиться и совершенствоваться . . . 31

7 Дополнительная литература **31**
- 7.1 Объяснения алгоритма PPO . . . 31
- 7.2 Особенности реализации PPO . . . 32
- 7.3 Выборка по значимости (importance sampling) . . . 32

8 Введение в PPO с использованием Sample Factory **32**

9 Практика: продвинутое глубокое RL — игра в Doom по пикселям с Sample Factory **33**
- 9.1 Модуль 8, часть 2: игра в Doom по пикселям с Sample Factory . . . 33
- 9.2 Настройка GPU . . . 34
- 9.3 Sample Factory . . . 35

9.4 ViZDoom . . . 36
9.5 Установка необходимых зависимостей ViZDoom . . . 37
9.6 Установка Sample Factory и ViZDoom . . . 37
9.7 Настройка среды Doom в Sample Factory . . . 37
9.8 Обучение агента . . . 39
9.9 Оценка обученной политики и запись видео . . . 39
9.10 Визуализация поведения агента . . . 39
9.11 Публикация контрольной точки и видео на Hugging Face Hub . . . 40
9.12 Загрузка другой модели . . . 41
9.13 Дополнительные задания: Doom Deathmatch . . . 41
9.14 Продолжайте учиться и совершенствоваться . . . 42

10 Заключение **42**
10.1 Продолжайте учиться и совершенствоваться . . . 43

1 Введение

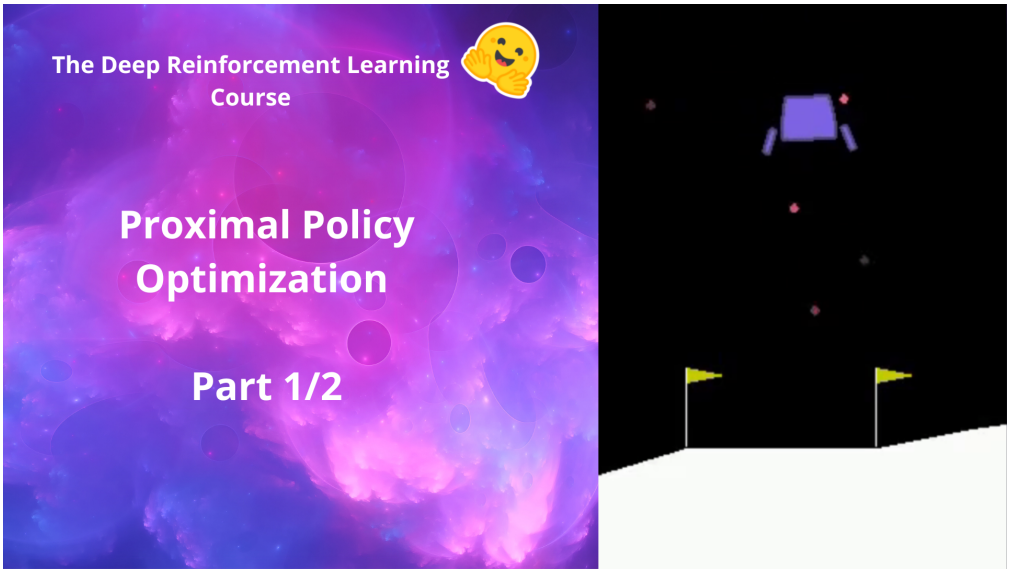


Рис. 1: Обложка модуля 8 курса Hugging Face Deep RL: алгоритм PPO [оригинал]

В модуле 6 мы познакомились с архитектурой Advantage Actor-Critic (A2C) — гибридным подходом, объединяющим методы на основе ценности и методы на основе политики. Он помогает стабилизировать обучение за счёт уменьшения дисперсии благодаря двум компонентам:

- *Актор (Actor)* определяет, **как ведёт себя агент**, то есть задаёт политику.
- *Критик (Critic)* оценивает, **насколько хорошим было выбранное действие**, используя метод на основе ценности.

Сегодня мы изучим Proximal Policy Optimization (PPO) — алгоритм, который **повышает устойчивость обучения агента, предотвращая чрезмерно большие обновления политики**. Для этого используется отношение вероятностей, показывающее различие между текущей и предыдущей политиками. Это отношение ограничивается интервалом $[1 - \epsilon, 1 + \epsilon]$.

Таким образом мы добиваемся того, **чтобы обновления политики не были слишком большими и обучение оставалось устойчивым**.

Модуль состоит из двух частей:

- В первой части вы изучите теорию PPO и реализуете собственного PPO-агента с нуля на основе реализации CleanRL. Чтобы проверить его устойчивость, вы используете среду LunarLander-v2. Именно LunarLander-v2 **была вашей первой средой в начале курса**. Тогда вы ещё не знали, как работает PPO, а теперь **сможете самостоятельно реализовать этот алгоритм и обучить агента**!
- Во второй части мы подробнее рассмотрим оптимизацию PPO с помощью Sample-Factory и обучим агента играть в ViZDoom — открытую реализацию игры Doom.



Рис. 2: Игровые среды ViZDoom, используемые для обучения агентов [оригинал]

Звучит интересно? Тогда приступим!

2 Интуиция, лежащая в основе PPO

Основная идея Proximal Policy Optimization (PPO) состоит в повышении устойчивости обучения политики за счёт ограничения её изменения на каждой эпохе: **мы хотим избежать слишком больших шагов обновления политики**.

Для этого есть две причины:

- Из практики известно, что небольшие обновления политики во время обучения **с большей вероятностью приводят к сходимости к оптимальному решению**.
- Слишком большой шаг обновления может привести к «падению с обрыва»: мы получим плохую политику, **после чего восстановление может занять очень много времени или даже стать невозможным**.



Рис. 3: Небольшие обновления политики повышают устойчивость обучения (адаптация пояснения PPO Джонатана Хуи) [оригинал]

Поэтому в PPO политика обновляется осторожно. Чтобы реализовать этот принцип, необходимо измерять, насколько текущая политика отличается от предыдущей. Для этого

рассчитывается отношение вероятностей действий при текущей и старой политиках. Затем отношение ограничивается диапазоном $[1-\epsilon, 1+\epsilon]$. Тем самым мы **устраняем стимул для новой политики слишком сильно отклоняться от старой** — отсюда и термин *proximal* в названии алгоритма.

3 Знакомство с суррогатной целевой функцией с ограничением

3.1 Повторение: целевая функция политики

Вспомним, какую целевую функцию мы оптимизировали в REINFORCE:

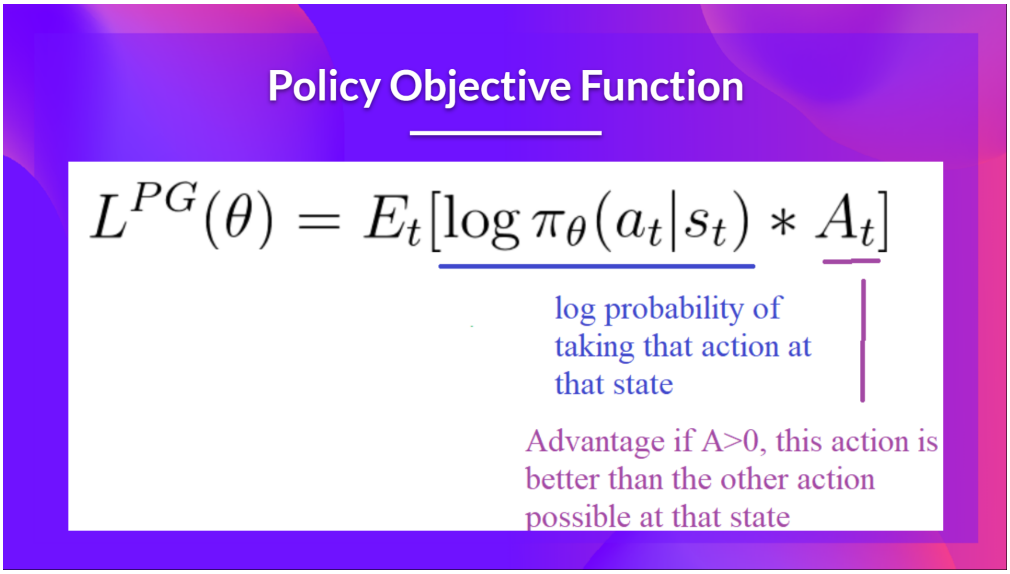


Рис. 4: Целевая функция градиента политики в алгоритме REINFORCE [оригинал]

Идея заключалась в том, что шаг градиентного подъёма по этой функции (эквивалентный градиентному спуску по её отрицанию) должен **побуждать агента выбирать действия, которые приносят более высокое вознаграждение, и избегать неудачных действий**.

Однако здесь возникает проблема с величиной шага:

- Если шаг слишком мал, **обучение идёт слишком медленно**.
- Если шаг слишком велик, **процесс обучения становится слишком нестабильным**.

В PPO обновление политики ограничивается с помощью новой целевой функции — *Clipped Surrogate Objective Function*, или суррогатной целевой функции с ограничением. Она **сдерживает изменение политики внутри небольшого диапазона посредством отсечения (clipping)**.

Эта функция **призвана предотвращать чрезмерно большие и разрушительные обновления весов**:

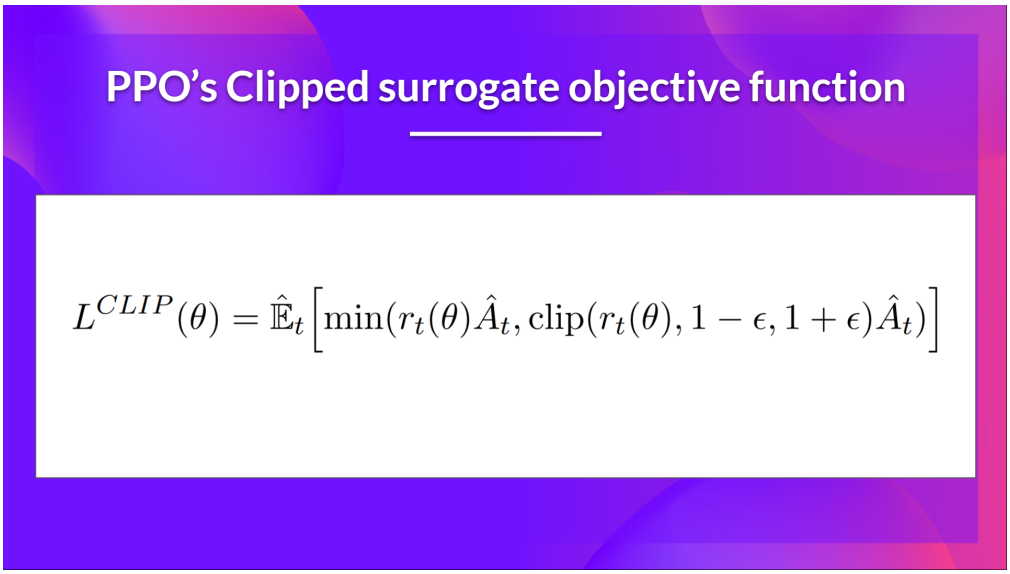


Рис. 5: Суррогатная целевая функция PPO с ограничением (clipping) [оригинал]

Разберём её по частям, чтобы понять принцип работы.

3.2 Функция отношения вероятностей

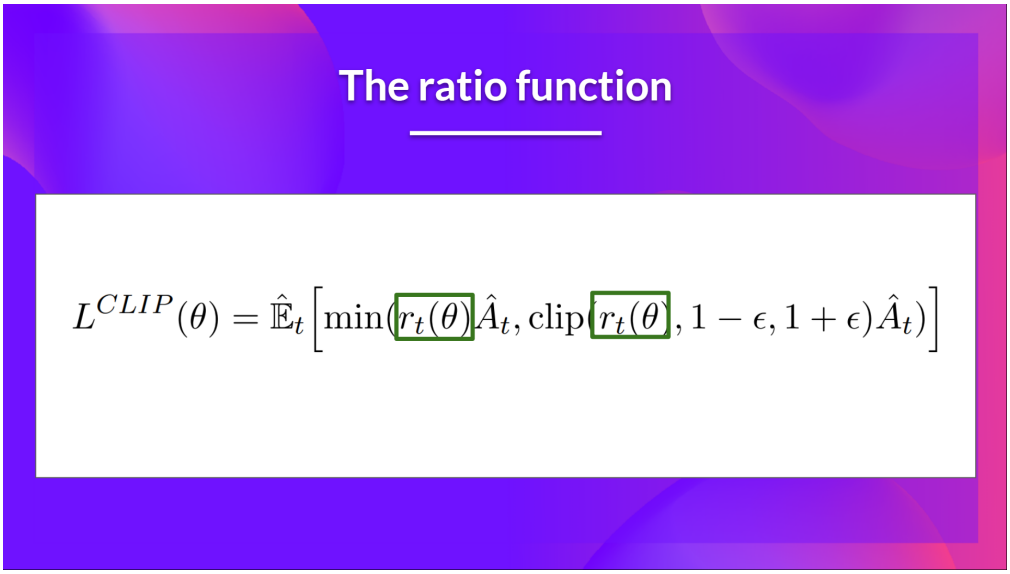


Рис. 6: Отношение вероятностей действия для старой и новой политик [оригинал]

Это отношение вычисляется следующим образом:

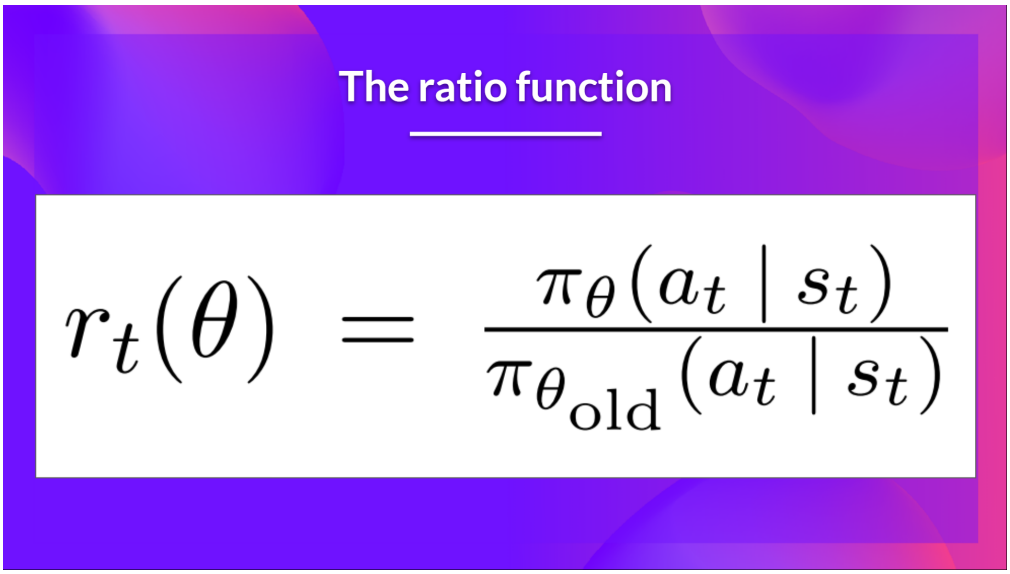


Рис. 7: Определение отношения вероятностей политик в PPO [оригинал]

В числителе находится вероятность выбрать действие a_t в состоянии s_t согласно текущей политике, а в знаменателе — вероятность того же действия согласно предыдущей политике.

Таким образом, $r_t(\theta)$ обозначает отношение вероятностей текущей и старой политик:

- Если $r_t(\theta) > 1$, то **действие a_t в состоянии s_t с большей вероятностью будет выбрано текущей политикой, чем старой**.
- Если $r_t(\theta)$ находится между 0 и 1, то **вероятность этого действия для текущей политики ниже, чем для старой**.

Следовательно, это отношение вероятностей позволяет **удобным способом оценивать различия между старой и текущей политиками**.

3.3 Часть суррогатной целевой функции без ограничения

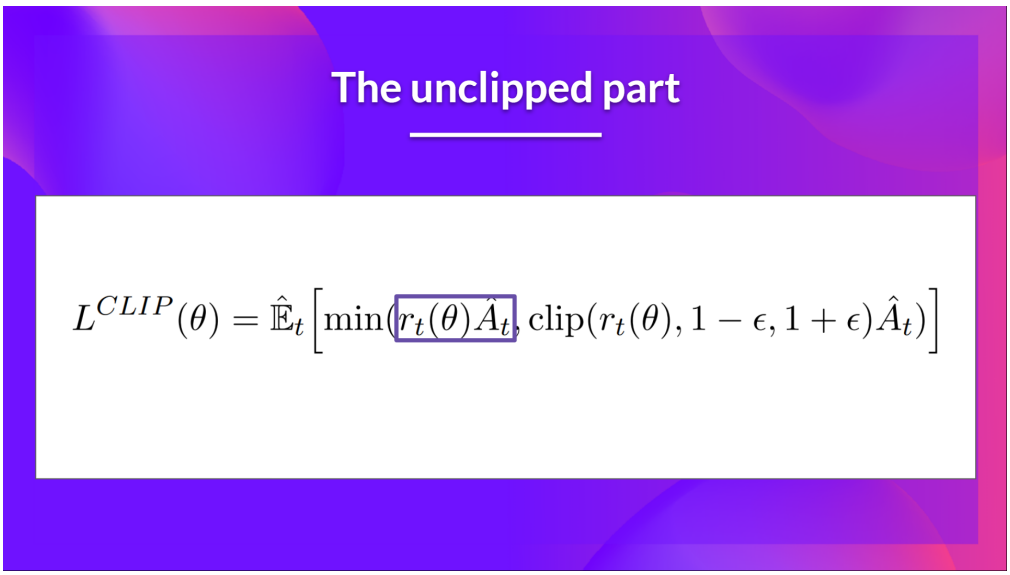


Рис. 8: Суррогатная целевая функция PPO без ограничения [оригинал]

Это отношение **может заменить логарифм вероятности, который мы использовали в целевой функции политики**. Так получается левая часть новой целевой функции: произведение отношения вероятностей на преимущество (advantage).

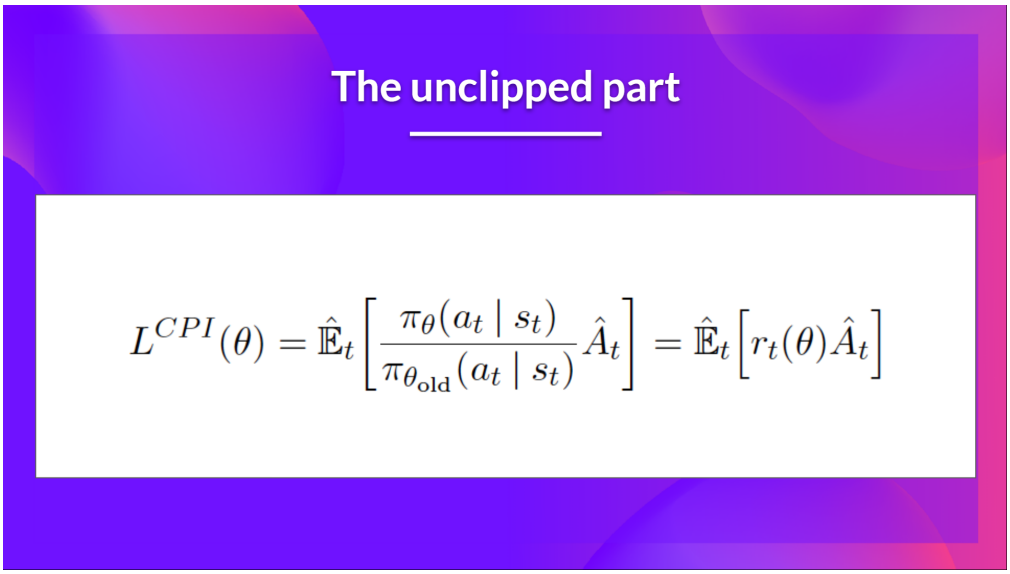


Рис. 9: Формула из статьи «Proximal Policy Optimization Algorithms» (https://arxiv.org/pdf/1707.06347.pdf) [оригинал]

Однако без дополнительных ограничений ситуация, когда выбранное действие стало намного вероятнее при текущей политике, чем при предыдущей, **приведёт к большому шагу градиента политики**, а значит, к **чрезмерному обновлению политики**.

3.4 Часть суррогатной целевой функции с ограничением

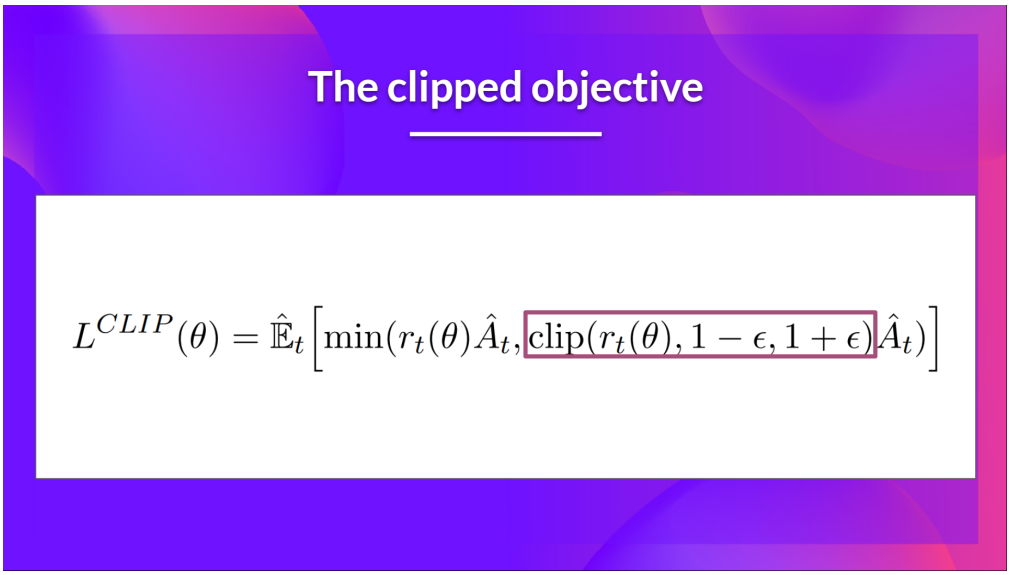


Рис. 10: Ограничение отношения вероятностей в PPO [оригинал]

Поэтому целевую функцию необходимо ограничить, штрафуя за изменения, при которых отношение вероятностей далеко уходит от единицы (в статье допустимый диапазон составляет от 0,8 до 1,2).

Ограничивая отношение вероятностей, мы не допускаем слишком большого обновления политики, поскольку текущая политика не может слишком сильно отличаться от старой.

Для этого существуют два подхода:

- *TRPO (Trust Region Policy Optimization)* накладывает ограничение через KL-дивергенцию вне целевой функции, сдерживая изменение политики. Однако **этот метод сложен в реализации и требует больших вычислительных затрат**.

- *PPO* ограничивает отношение вероятностей непосредственно внутри целевой функции посредством **суррогатной функции с отсечением**.

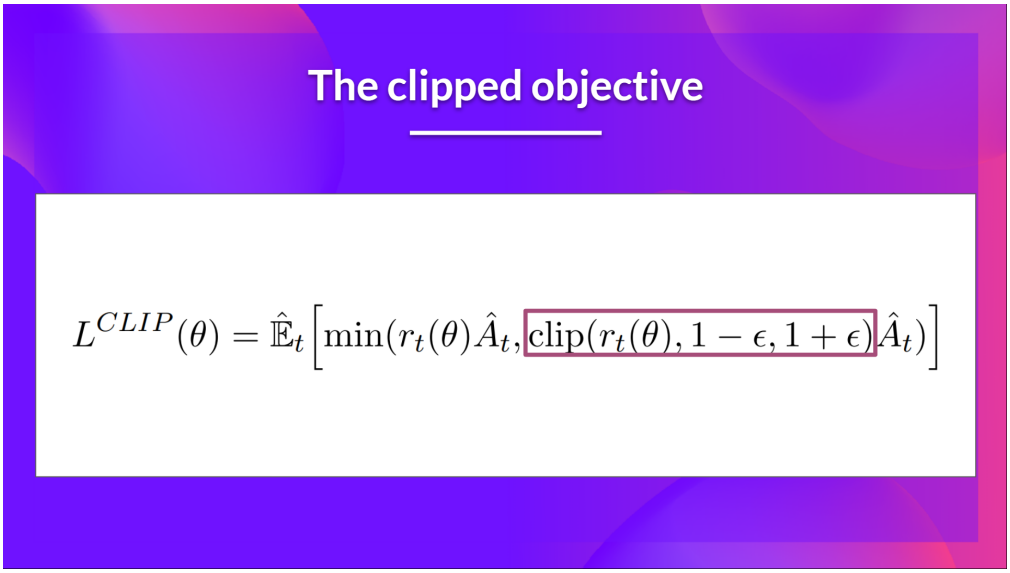


Рис. 11: Ограничение отношения вероятностей в PPO [оригинал]

В ограниченной части функции величина $r_t(\theta)$ отсекается в пределах $[1-\epsilon, 1+\epsilon]$.

Таким образом, в Clipped Surrogate Objective Function присутствуют два отношения вероятностей: исходное без ограничения и ограниченное интервалом $[1-\epsilon, 1+\epsilon]$. Параметр ϵ — гиперпараметр, задающий ширину интервала отсечения (в статье $\epsilon = 0.2$).

Затем мы выбираем минимум между ограниченной и неограниченной целевыми функциями. **Итоговая функция тем самым задаёт нижнюю, или пессимистическую, оценку неограниченной функции**.

Взятие минимума означает, что **в зависимости от отношения вероятностей и значения преимущества будет выбрана либо ограниченная, либо неограниченная целевая функция**.

4 Визуализация суррогатной целевой функции с ограничением

Не переживайте: **если сейчас всё кажется сложным, это совершенно нормально**. Давайте посмотрим, как выглядит эта суррогатная функция с ограничением, чтобы нагляднее понять происходящее.

	$p_t(\theta) > 0$	A_t	**Return Value of** *min*	**Objective is Clipped**	**Sign of Objective**	**Gradient**
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1-\epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1-\epsilon$	−	$(1-\epsilon)A_t$	yes	−	**0**
5	$p_t(\theta) > 1+\epsilon$	+	$(1+\epsilon)A_t$	yes	+	**0**
6	$p_t(\theta) > 1+\epsilon$	−	$p_t(\theta)A_t$	no	−	✓

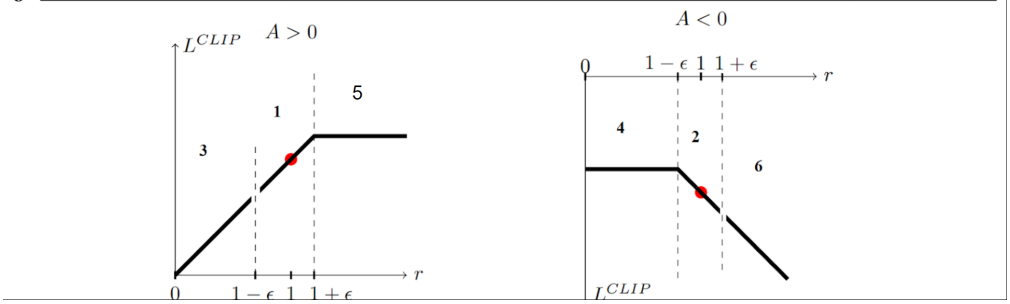


Рис. 12: Таблица шести случаев PPO из работы Даниэля Бика «Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization» (https://fse.studenttheses.ub.rug.nl/25709/1/mAI_2021_BickD.pdf) [оригинал]

Рассмотрим шесть различных ситуаций. Для начала вспомним, что мы всегда берём минимум из ограниченной и неограниченной целевых функций.

4.1 Случаи 1 и 2: отношение находится внутри интервала

В случаях 1 и 2 **отсечение не срабатывает, поскольку отношение лежит внутри интервала** $[1-\epsilon, 1+\epsilon]$.

В случае 1 преимущество положительное: **действие лучше среднего** среди возможных действий в данном состоянии. Поэтому следует поощрять текущую политику повышать вероятность этого действия в данном состоянии.

Поскольку отношение находится в допустимом интервале, **мы можем увеличить вероятность выбора данного действия текущей политикой**.

В случае 2 преимущество отрицательное: действие хуже среднего среди действий в данном состоянии. Следовательно, нужно поощрять текущую политику выбирать это действие реже.

Поскольку отношение находится в допустимом интервале, **мы можем снизить вероятность выбора данного действия текущей политикой**.

4.2 Случаи 3 и 4: отношение ниже интервала

	$p_t(\theta) > 0$	A_t	Return Value of *min*	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1-\epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1-\epsilon$	−	$(1-\epsilon)A_t$	yes	−	**0**
5	$p_t(\theta) > 1+\epsilon$	+	$(1+\epsilon)A_t$	yes	+	**0**
6	$p_t(\theta) > 1+\epsilon$	−	$p_t(\theta)A_t$	no	−	✓

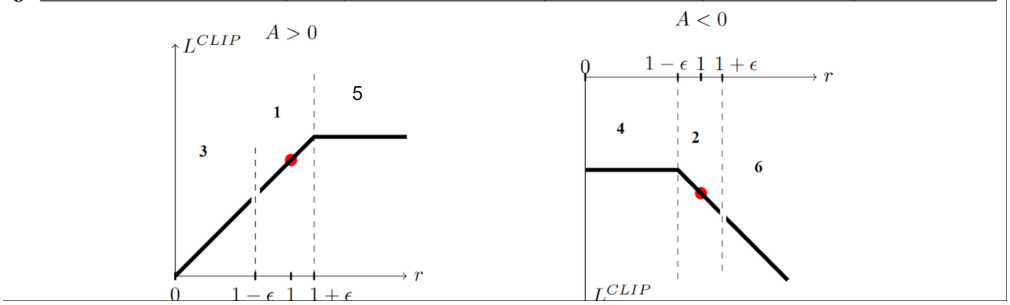


Рис. 13: Таблица шести случаев PPO из работы Даниэля Бика «Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization» (https://fse.studenttheses.ub.rug.nl/25709/1/mAI_2021_BickD.pdf) [оригинал]

Если отношение вероятностей меньше $[1-\epsilon]$, то вероятность выбора данного действия в данном состоянии заметно ниже, чем у старой политики.

Если, как в случае 3, оценка преимущества положительна (A>0), то **вероятность выбора данного действия следует увеличить**.

Но если, как в случае 4, преимущество отрицательно, **мы не хотим дополнительно уменьшать** вероятность этого действия в данном состоянии. Поэтому градиент равен нулю (функция здесь постоянна), и веса не обновляются.

4.3 Случаи 5 и 6: отношение выше интервала

	$p_t(\theta) > 0$	A_t	Return Value of *min*	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1-\epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1-\epsilon$	−	$(1-\epsilon)A_t$	yes	−	**0**
5	$p_t(\theta) > 1+\epsilon$	+	$(1+\epsilon)A_t$	yes	+	**0**
6	$p_t(\theta) > 1+\epsilon$	−	$p_t(\theta)A_t$	no	−	✓

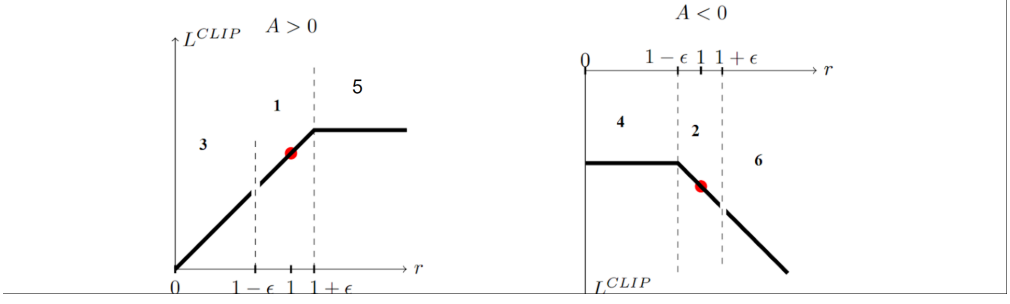


Рис. 14: Таблица шести случаев PPO из работы Даниэля Бика «Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization» (https://fse.studenttheses.ub.rug.nl/25709/1/mAI_2021_BickD.pdf) [оригинал]

Если отношение вероятностей больше $[1+\epsilon]$, то вероятность этого действия в текущей политике **намного выше, чем в предыдущей**.

Если, как в случае 5, преимущество положительно, **не следует слишком сильно повышать вероятность действия**. Она уже выше, чем в старой политике. Поэтому градиент равен нулю (на этом участке функция постоянна), и веса не обновляются.

Если, как в случае 6, преимущество отрицательно, мы хотим уменьшить вероятность выбора данного действия в данном состоянии.

Подведём итог: **мы обновляем политику только тогда, когда минимум соответствует неограниченной части целевой функции**. Когда минимум соответствует ограниченной части, градиент равен нулю, поэтому веса политики не изменяются.

Таким образом, политика обновляется лишь при выполнении одного из условий:

- Отношение находится внутри интервала $[1-\epsilon, 1+\epsilon]$.
- Отношение находится вне интервала, но **знак преимущества способствует возвращению отношения к интервалу**:
  - Отношение ниже нижней границы, а преимущество > 0.
  - Отношение выше верхней границы, а преимущество < 0.

Может возникнуть вопрос, почему при выборе ограниченной части функции градиент равен нулю. Когда отношение отсекается, дифференцируется не выражение $r_t(\theta) * A_t$, а $(1-\epsilon) * A_t$ либо $(1+\epsilon) * A_t$. Оба эти выражения не зависят от θ, поэтому их производные равны нулю.

Итак, благодаря суррогатной целевой функции с отсечением **мы ограничиваем возможное отклонение текущей политики от старой**. Когда отношение вероятностей выходит за заданный интервал, отсечение устраняет стимул к дальнейшему увеличению отклонения: градиент становится нулевым. При отношении $> 1+\epsilon$ или $< 1-\epsilon$ градиент соответствующей ограниченной части равен нулю.

Окончательная функция потерь PPO в архитектуре Actor–Critic объединяет суррогатную целевую функцию с отсечением, функцию потерь оценки ценности и бонус за энтропию:

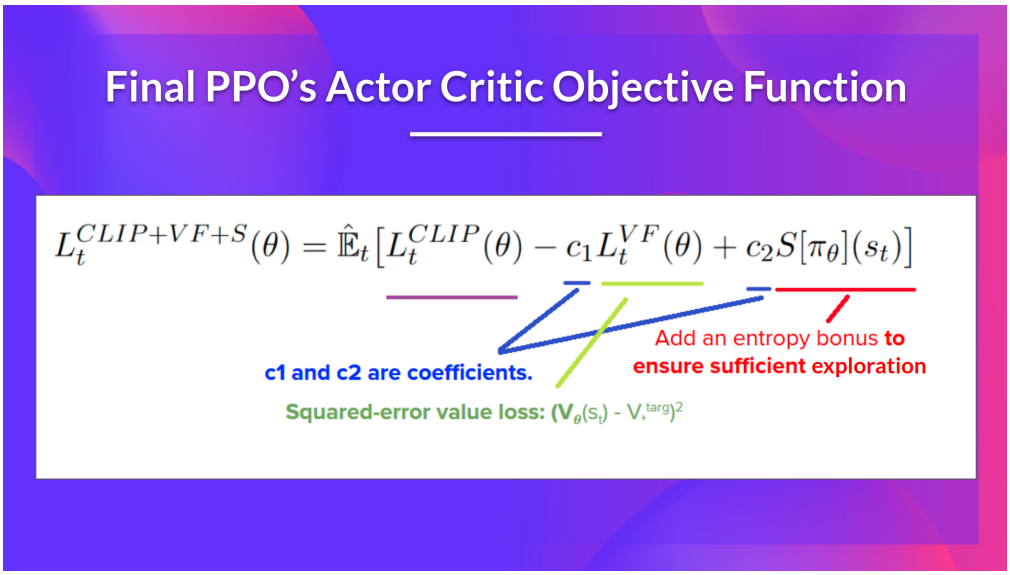


Рис. 15: Итоговая функция потерь Actor–Critic PPO: политика, ценность и энтропия [оригинал]

Материал непростой. Уделите время разбору этих ситуаций по таблице и графику. **Важно понять, почему именно такая конструкция имеет смысл.** Для более глубокого изучения полезна работа Даниэля Бика «Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization», особенно раздел 3.4.

5 Практическая часть

Исходный ноутбук Google Colab

Теперь, когда мы изучили теорию PPO, лучший способ разобраться в алгоритме — **реализовать его с нуля**.

Реализация архитектуры с нуля — лучший способ понять её устройство и полезная привычка. Мы уже делали это для метода на основе ценности (Q-Learning) и метода на основе политики (REINFORCE).

Для реализации PPO мы воспользуемся двумя источниками:

- Руководство Косты Хуанга (Costa Huang). Коста — автор CleanRL, библиотеки глубокого обучения с подкреплением с качественными реализациями алгоритмов в отдельных файлах и возможностями, удобными для научных исследований.
- Помимо руководства, для углублённого изучения прочитайте материал о 13 основных особенностях реализации: https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/.

Чтобы проверить устойчивость агента, мы обучим его в следующей среде:

- LunarLander-v2

Посмотреть оригинальное видео из курса

В конце мы опубликуем обученную модель на Hub, оценим её и посмотрим, как играет агент.

LunarLander-v2 — первая среда, с которой вы работали в начале курса. Тогда вы ещё не знали, как устроен алгоритм, а теперь сможете самостоятельно его реализовать и обучить. **Разве это не здорово!**

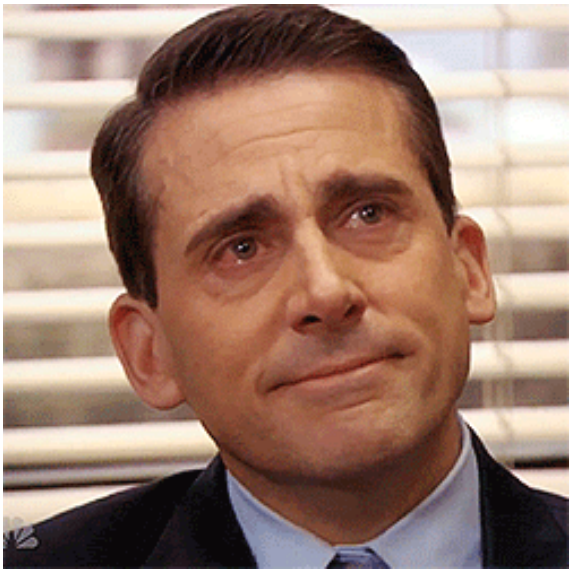


Рис. 16: Кадр анимации из исходного курса [оригинальная анимация]

Приступим!

Ноутбук Google Colab:

Открыть исходный ноутбук в Colab

5.1 Модуль 8: PPO на PyTorch

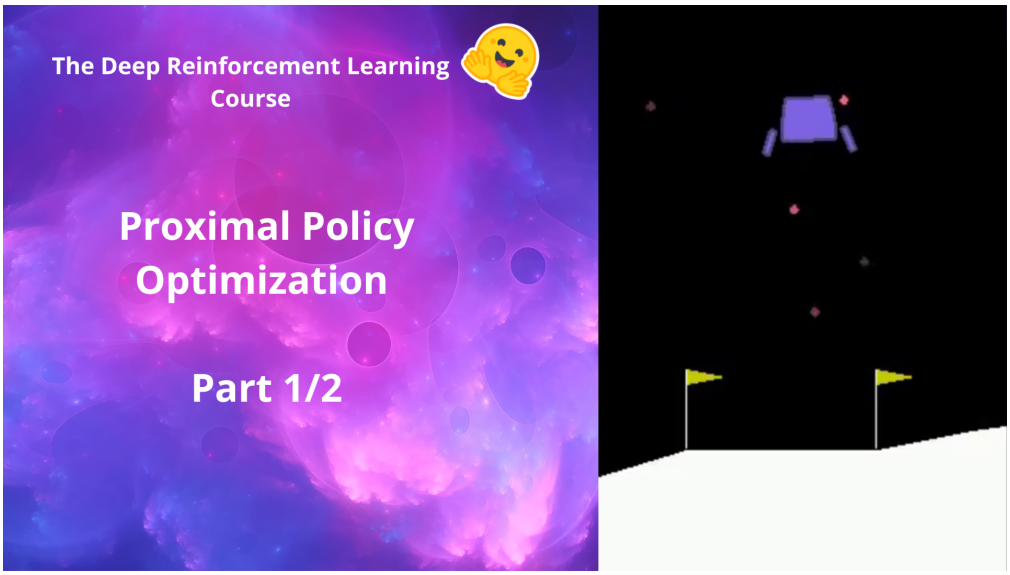


Рис. 17: Обложка модуля 8 курса Hugging Face Deep RL: алгоритм PPO [оригинал]

В этом ноутбуке вы научитесь **самостоятельно реализовывать PPO-агента на PyTorch, используя реализацию CleanRL в качестве образца**.

Чтобы проверить его устойчивость, мы обучим агента в следующей среде:

- LunarLander-v2

Мы постоянно совершенствуем наши учебные материалы. Поэтому, **если вы заметите проблемы в этом ноутбуке**, пожалуйста, создайте обращение в репозитории GitHub.

5.2 Цели практического ноутбука

После выполнения ноутбука вы сможете:

- **Самостоятельно реализовать PPO-агента на PyTorch**.
- **Опубликовать обученного агента и исходный код на Hugging Face Hub**, добавив видеозапись его поведения и результат оценки.

5.3 Предварительные требования

Перед началом работы с ноутбуком необходимо:

[ ] Изучить алгоритм PPO в теоретической части модуля 8.

Для зачёта этого практического задания в рамках сертификации нужно загрузить одну модель. Минимального результата не требуется, однако мы **рекомендуем пробовать разные наборы гиперпараметров, чтобы улучшить качество обучения**.

Если вы не видите свою модель, **прокрутите страницу до конца и нажмите кнопку обновления**.

Подробнее о сертификации читайте в соответствующем разделе: https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process.

5.4 Настройка GPU

- Чтобы **ускорить обучение агента, будем использовать GPU**. В Google Colab откройте меню `Runtime > Change Runtime type`.

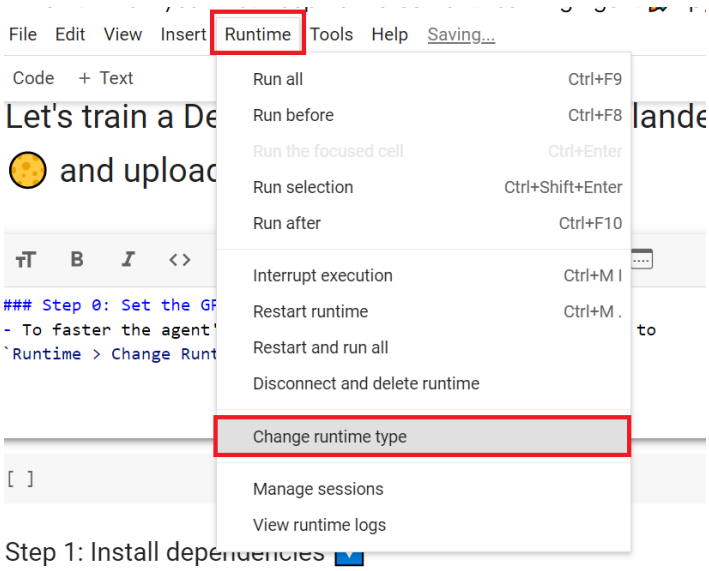


Рис. 18: Меню Google Colab для выбора среды выполнения [оригинал]

- `Hardware Accelerator > GPU`

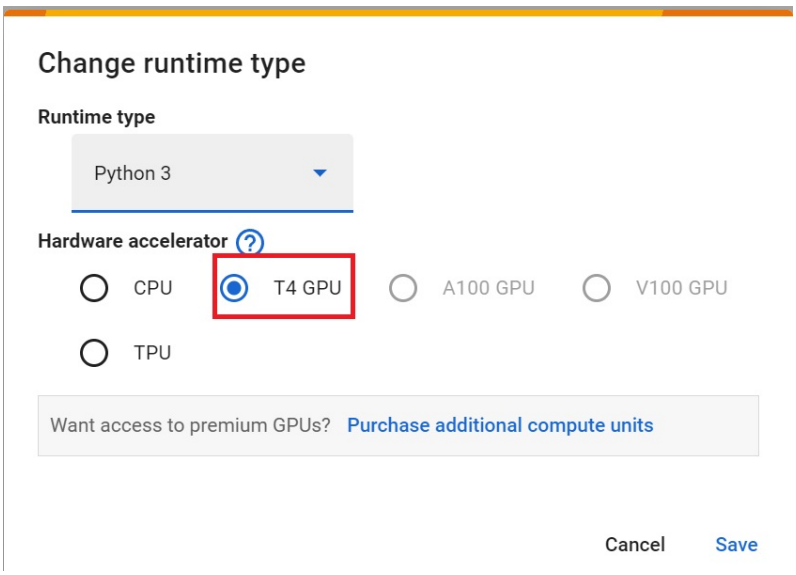


Рис. 19: Выбор GPU-ускорителя в Google Colab [оригинал]

5.5 Создание виртуального экрана

Во время работы с ноутбуком нам потребуется записать видео с поведением агента. Для этого в Colab **нужен виртуальный экран, на котором будет отображаться среда** и с которого можно будет получать кадры.

Следующая ячейка устанавливает библиотеки, создаёт виртуальный экран и запускает его.

```
apt install python-opengl
apt install ffmpeg
apt install xvfb
pip install pyglet==1.5
pip install pyvirtualdisplay
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
```

```
virtual_display.start()
```

5.6 Установка зависимостей

В этом упражнении используется `gym==0.21`, поскольку видеоролик был записан с библиотекой Gym.

```
pip install gym==0.22
pip install imageio-ffmpeg
pip install huggingface_hub
pip install gym[box2d]==0.22
```

5.7 Реализуем PPO с нуля по руководству Косты Хуанга

- Для основной реализации PPO мы воспользуемся отличным руководством Косты Хуанга (Costa Huang).
- Чтобы глубже разобраться в деталях, дополнительно можно изучить 37 основных особенностей реализации: https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/.

Видеоурок: https://youtu.be/MEt6rrxH8W4.

```
from IPython.display import HTML

HTML(
    '<iframe width="560" height="315" src="https://www.youtube.com/embed/MEt6rrxH8W4" title="
    YouTube video player" frameborder="0" allow="accelerometer; autoplay; clipboard-write;
    encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>'
)
```

5.8 Добавление интеграции с Hugging Face

- Чтобы загрузить нашу модель на Hub, сначала определим функцию `package_to_hub`.
- Добавим зависимости, необходимые для публикации модели на Hub.

```
from huggingface_hub import HfApi, upload_folder
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import tempfile
import json
import shutil
import imageio

from wasabi import Printer

msg = Printer()
```

- Добавим новый аргумент в функцию `parse_args()`, чтобы задать идентификатор репозитория (repo ID), в который будет опубликована модель.

```
# Adding HuggingFace argument
parser.add_argument(
    "--repo-id",
    type=str,
```

```
    default="ThomasSimonini/ppo-CartPole-v1",
    help="id of the model repository from the Hugging Face Hub {username/repo_name}",
)
```

- Затем добавим методы, необходимые для публикации модели на Hub.
- Эти методы выполняют следующие задачи:
  - `_evalutate_agent()`: оценивает качество агента.
  - `_generate_model_card()`: создаёт карточку модели агента.
  - `_record_video()`: записывает видео с поведением агента.

```
def package_to_hub(
    repo_id,
    model,
    hyperparameters,
    eval_env,
    video_fps=30,
    commit_message="Push agent to the Hub",
    token=None,
    logs=None,
):
    """
    Evaluate, Generate a video and Upload a model to Hugging Face Hub.
    This method does the complete pipeline:
    - It evaluates the model
    - It generates the model card
    - It generates a replay video of the agent
    - It pushes everything to the hub
    :param repo_id: id of the model repository from the Hugging Face Hub
    :param model: trained model
    :param eval_env: environment used to evaluate the agent
    :param fps: number of fps for rendering the video
    :param commit_message: commit message
    :param logs: directory on local machine of tensorboard logs you'd like to upload
    """
    msg.info(
        "This function will save, evaluate, generate a video of your agent, "
        "create a model card and push everything to the hub. "
        "It might take up to 1min. \n "
        "This is a work in progress: if you encounter a bug, please open an issue."
    )
    # Step 1: Clone or create the repo
    repo_url = HfApi().create_repo(
        repo_id=repo_id,
        token=token,
        private=False,
        exist_ok=True,
    )

    with tempfile.TemporaryDirectory() as tmpdirname:
        tmpdirname = Path(tmpdirname)

        # Step 2: Save the model
        torch.save(model.state_dict(), tmpdirname / "model.pt")

        # Step 3: Evaluate the model and build JSON
        mean_reward, std_reward = _evaluate_agent(eval_env, 10, model)

        # First get datetime
        eval_datetime = datetime.datetime.now()
        eval_form_datetime = eval_datetime.isoformat()
```

```
        evaluate_data = {
            "env_id": hyperparameters.env_id,
            "mean_reward": mean_reward,
            "std_reward": std_reward,
            "n_evaluation_episodes": 10,
            "eval_datetime": eval_form_datetime,
        }

        # Write a JSON file
        with open(tmpdirname / "results.json", "w") as outfile:
            json.dump(evaluate_data, outfile)

        # Step 4: Generate a video
        video_path = tmpdirname / "replay.mp4"
        record_video(eval_env, model, video_path, video_fps)

        # Step 5: Generate the model card
        generated_model_card, metadata = _generate_model_card(
            "PPO", hyperparameters.env_id, mean_reward, std_reward, hyperparameters
        )
        _save_model_card(tmpdirname, generated_model_card, metadata)

        # Step 6: Add logs if needed
        if logs:
            _add_logdir(tmpdirname, Path(logs))

        msg.info(f"Pushing repo {repo_id} to the Hugging Face Hub")

        repo_url = upload_folder(
            repo_id=repo_id,
            folder_path=tmpdirname,
            path_in_repo="",
            commit_message=commit_message,
            token=token,
        )

        msg.info(f"Your model is pushed to the Hub. You can view your model here: {repo_url}")
    return repo_url


def _evaluate_agent(env, n_eval_episodes, policy):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and std of
    reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param policy: The agent
    """
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state = env.reset()
        step = 0
        done = False
        total_rewards_ep = 0

        while done is False:
            state = torch.Tensor(state).to(device)
            action, _, _, _ = policy.get_action_and_value(state)
            new_state, reward, done, info = env.step(action.cpu().numpy())
            total_rewards_ep += reward
            if done:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
```

```
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward


def record_video(env, policy, out_directory, fps=30):
    images = []
    done = False
    state = env.reset()
    img = env.render(mode="rgb_array")
    images.append(img)
    while not done:
        state = torch.Tensor(state).to(device)
        # Take the action (index) that have the maximum expected future reward given that state
        action, _, _, _ = policy.get_action_and_value(state)
        state, reward, done, info = env.step(
            action.cpu().numpy()
        )  # We directly put next_state = state for recording logic
        img = env.render(mode="rgb_array")
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=fps)


def _generate_model_card(model_name, env_id, mean_reward, std_reward, hyperparameters):
    """
    Generate the model card for the Hub
    :param model_name: name of the model
    :env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    :hyperparameters: training arguments
    """
    # Step 1: Select the tags
    metadata = generate_metadata(model_name, env_id, mean_reward, std_reward)

    # Transform the hyperparams namespace to string
    converted_dict = vars(hyperparameters)
    converted_str = str(converted_dict)
    converted_str = converted_str.split(", ")
    converted_str = "\n".join(converted_str)

    # Step 2: Generate the model card
    model_card = f"""
  # PPO Agent Playing {env_id}

  This is a trained model of a PPO agent playing {env_id}.

  # Hyperparameters
  """
    return model_card, metadata


def generate_metadata(model_name, env_id, mean_reward, std_reward):
    """
    Define the tags for the model card
    :param model_name: name of the model
    :param env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    """
    metadata = {}
    metadata["tags"] = [
        env_id,
        "ppo",
        "deep-reinforcement-learning",
```

```
        "reinforcement-learning",
        "custom-implementation",
        "deep-rl-course",
    ]

    # Add metrics
    eval = metadata_eval_result(
        model_pretty_name=model_name,
        task_pretty_name="reinforcement-learning",
        task_id="reinforcement-learning",
        metrics_pretty_name="mean_reward",
        metrics_id="mean_reward",
        metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
        dataset_pretty_name=env_id,
        dataset_id=env_id,
    )

    # Merges both dictionaries
    metadata = {**metadata, **eval}

    return metadata


def _save_model_card(local_path, generated_model_card, metadata):
    """Saves a model card for the repository.
    :param local_path: repository directory
    :param generated_model_card: model card generated by _generate_model_card()
    :param metadata: metadata
    """
    readme_path = local_path / "README.md"
    readme = ""
    if readme_path.exists():
        with readme_path.open("r", encoding="utf8") as f:
            readme = f.read()
    else:
        readme = generated_model_card

    with readme_path.open("w", encoding="utf-8") as f:
        f.write(readme)

    # Save our metrics to Readme metadata
    metadata_save(readme_path, metadata)


def _add_logdir(local_path: Path, logdir: Path):
    """Adds a logdir to the repository.
    :param local_path: repository directory
    :param logdir: logdir directory
    """
    if logdir.exists() and logdir.is_dir():
        # Add the logdir to the repository under new dir called logs
        repo_logdir = local_path / "logs"

        # Delete current logs if they exist
        if repo_logdir.exists():
            shutil.rmtree(repo_logdir)

        # Copy logdir into repo logdir
        shutil.copytree(logdir, repo_logdir)
```

- Наконец, вызовем эту функцию в конце обучения PPO.

```
# Create the evaluation environment
```

```
eval_env = gym.make(args.env_id)

package_to_hub(
    repo_id=args.repo_id,
    model=agent,  # The model we want to save
    hyperparameters=args,
    eval_env=gym.make(args.env_id),
    logs=f"runs/{run_name}",
)
```

- Вот как выглядит окончательный файл ppo.py:

```
# docs and experiment results can be found at https://docs.cleanrl.dev/rl-algorithms/ppo/#ppopy

import argparse
import os
import random
import time
from distutils.util import strtobool

import gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions.categorical import Categorical
from torch.utils.tensorboard import SummaryWriter

from huggingface_hub import HfApi, upload_folder
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import tempfile
import json
import shutil
import imageio

from wasabi import Printer

msg = Printer()


def parse_args():
    # fmt: off
    parser = argparse.ArgumentParser()
    parser.add_argument("--exp-name", type=str, default=os.path.basename(__file__).rstrip(".py"),
        help="the name of this experiment")
    parser.add_argument("--seed", type=int, default=1,
        help="seed of the experiment")
    parser.add_argument("--torch-deterministic", type=lambda x: bool(strtobool(x)), default=True,
    nargs="?", const=True,
        help="if toggled, `torch.backends.cudnn.deterministic=False`")
    parser.add_argument("--cuda", type=lambda x: bool(strtobool(x)), default=True, nargs="?", const
    =True,
        help="if toggled, cuda will be enabled by default")
    parser.add_argument("--track", type=lambda x: bool(strtobool(x)), default=False, nargs="?",
    const=True,
        help="if toggled, this experiment will be tracked with Weights and Biases")
    parser.add_argument("--wandb-project-name", type=str, default="cleanRL",
        help="the wandb's project name")
    parser.add_argument("--wandb-entity", type=str, default=None,
        help="the entity (team) of wandb's project")
```

```
    parser.add_argument("--capture-video", type=lambda x: bool(strtobool(x)), default=False, nargs
    ="?", const=True,
        help="weather to capture videos of the agent performances (check out `videos` folder)")

    # Algorithm specific arguments
    parser.add_argument("--env-id", type=str, default="CartPole-v1",
        help="the id of the environment")
    parser.add_argument("--total-timesteps", type=int, default=50000,
        help="total timesteps of the experiments")
    parser.add_argument("--learning-rate", type=float, default=2.5e-4,
        help="the learning rate of the optimizer")
    parser.add_argument("--num-envs", type=int, default=4,
        help="the number of parallel game environments")
    parser.add_argument("--num-steps", type=int, default=128,
        help="the number of steps to run in each environment per policy rollout")
    parser.add_argument("--anneal-lr", type=lambda x: bool(strtobool(x)), default=True, nargs="?",
    const=True,
        help="Toggle learning rate annealing for policy and value networks")
    parser.add_argument("--gae", type=lambda x: bool(strtobool(x)), default=True, nargs="?", const=
    True,
        help="Use GAE for advantage computation")
    parser.add_argument("--gamma", type=float, default=0.99,
        help="the discount factor gamma")
    parser.add_argument("--gae-lambda", type=float, default=0.95,
        help="the lambda for the general advantage estimation")
    parser.add_argument("--num-minibatches", type=int, default=4,
        help="the number of mini-batches")
    parser.add_argument("--update-epochs", type=int, default=4,
        help="the K epochs to update the policy")
    parser.add_argument("--norm-adv", type=lambda x: bool(strtobool(x)), default=True, nargs="?",
    const=True,
        help="Toggles advantages normalization")
    parser.add_argument("--clip-coef", type=float, default=0.2,
        help="the surrogate clipping coefficient")
    parser.add_argument("--clip-vloss", type=lambda x: bool(strtobool(x)), default=True, nargs="?",
     const=True,
        help="Toggles whether or not to use a clipped loss for the value function, as per the paper
    .")
    parser.add_argument("--ent-coef", type=float, default=0.01,
        help="coefficient of the entropy")
    parser.add_argument("--vf-coef", type=float, default=0.5,
        help="coefficient of the value function")
    parser.add_argument("--max-grad-norm", type=float, default=0.5,
        help="the maximum norm for the gradient clipping")
    parser.add_argument("--target-kl", type=float, default=None,
        help="the target KL divergence threshold")

    # Adding HuggingFace argument
    parser.add_argument("--repo-id", type=str, default="ThomasSimonini/ppo-CartPole-v1", help="id
    of the model repository from the Hugging Face Hub {username/repo_name}")

    args = parser.parse_args()
    args.batch_size = int(args.num_envs * args.num_steps)
    args.minibatch_size = int(args.batch_size // args.num_minibatches)
    # fmt: on
    return args


def package_to_hub(
    repo_id,
    model,
    hyperparameters,
    eval_env,
    video_fps=30,
    commit_message="Push agent to the Hub",
```

```
    token=None,
    logs=None,
):
    """
    Evaluate, Generate a video and Upload a model to Hugging Face Hub.
    This method does the complete pipeline:
    - It evaluates the model
    - It generates the model card
    - It generates a replay video of the agent
    - It pushes everything to the hub
    :param repo_id: id of the model repository from the Hugging Face Hub
    :param model: trained model
    :param eval_env: environment used to evaluate the agent
    :param fps: number of fps for rendering the video
    :param commit_message: commit message
    :param logs: directory on local machine of tensorboard logs you'd like to upload
    """
    msg.info(
        "This function will save, evaluate, generate a video of your agent, "
        "create a model card and push everything to the hub. "
        "It might take up to 1min. \n "
        "This is a work in progress: if you encounter a bug, please open an issue."
    )
    # Step 1: Clone or create the repo
    repo_url = HfApi().create_repo(
        repo_id=repo_id,
        token=token,
        private=False,
        exist_ok=True,
    )

    with tempfile.TemporaryDirectory() as tmpdirname:
        tmpdirname = Path(tmpdirname)

        # Step 2: Save the model
        torch.save(model.state_dict(), tmpdirname / "model.pt")

        # Step 3: Evaluate the model and build JSON
        mean_reward, std_reward = _evaluate_agent(eval_env, 10, model)

        # First get datetime
        eval_datetime = datetime.datetime.now()
        eval_form_datetime = eval_datetime.isoformat()

        evaluate_data = {
            "env_id": hyperparameters.env_id,
            "mean_reward": mean_reward,
            "std_reward": std_reward,
            "n_evaluation_episodes": 10,
            "eval_datetime": eval_form_datetime,
        }

        # Write a JSON file
        with open(tmpdirname / "results.json", "w") as outfile:
            json.dump(evaluate_data, outfile)

        # Step 4: Generate a video
        video_path = tmpdirname / "replay.mp4"
        record_video(eval_env, model, video_path, video_fps)

        # Step 5: Generate the model card
        generated_model_card, metadata = _generate_model_card(
            "PPO", hyperparameters.env_id, mean_reward, std_reward, hyperparameters
        )
        _save_model_card(tmpdirname, generated_model_card, metadata)
```

```
        # Step 6: Add logs if needed
        if logs:
            _add_logdir(tmpdirname, Path(logs))

        msg.info(f"Pushing repo {repo_id} to the Hugging Face Hub")

        repo_url = upload_folder(
            repo_id=repo_id,
            folder_path=tmpdirname,
            path_in_repo="",
            commit_message=commit_message,
            token=token,
        )

        msg.info(f"Your model is pushed to the Hub. You can view your model here: {repo_url}")
    return repo_url


def _evaluate_agent(env, n_eval_episodes, policy):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and std of
    reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param policy: The agent
    """
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state = env.reset()
        step = 0
        done = False
        total_rewards_ep = 0

        while done is False:
            state = torch.Tensor(state).to(device)
            action, _, _, _ = policy.get_action_and_value(state)
            new_state, reward, done, info = env.step(action.cpu().numpy())
            total_rewards_ep += reward
            if done:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward


def record_video(env, policy, out_directory, fps=30):
    images = []
    done = False
    state = env.reset()
    img = env.render(mode="rgb_array")
    images.append(img)
    while not done:
        state = torch.Tensor(state).to(device)
        # Take the action (index) that have the maximum expected future reward given that state
        action, _, _, _ = policy.get_action_and_value(state)
        state, reward, done, info = env.step(
            action.cpu().numpy()
        )  # We directly put next_state = state for recording logic
        img = env.render(mode="rgb_array")
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=fps)
```

```
def _generate_model_card(model_name, env_id, mean_reward, std_reward, hyperparameters):
    """
    Generate the model card for the Hub
    :param model_name: name of the model
    :env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    :hyperparameters: training arguments
    """
    # Step 1: Select the tags
    metadata = generate_metadata(model_name, env_id, mean_reward, std_reward)

    # Transform the hyperparams namespace to string
    converted_dict = vars(hyperparameters)
    converted_str = str(converted_dict)
    converted_str = converted_str.split(", ")
    converted_str = "\n".join(converted_str)

    # Step 2: Generate the model card
    model_card = f"""
  # PPO Agent Playing {env_id}

  This is a trained model of a PPO agent playing {env_id}.

  # Hyperparameters
  """
    return model_card, metadata


def generate_metadata(model_name, env_id, mean_reward, std_reward):
    """
    Define the tags for the model card
    :param model_name: name of the model
    :param env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    """
    metadata = {}
    metadata["tags"] = [
        env_id,
        "ppo",
        "deep-reinforcement-learning",
        "reinforcement-learning",
        "custom-implementation",
        "deep-rl-course",
    ]

    # Add metrics
    eval = metadata_eval_result(
        model_pretty_name=model_name,
        task_pretty_name="reinforcement-learning",
        task_id="reinforcement-learning",
        metrics_pretty_name="mean_reward",
        metrics_id="mean_reward",
        metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
        dataset_pretty_name=env_id,
        dataset_id=env_id,
    )

    # Merges both dictionaries
    metadata = {**metadata, **eval}

    return metadata
```

```
def _save_model_card(local_path, generated_model_card, metadata):
    """Saves a model card for the repository.
    :param local_path: repository directory
    :param generated_model_card: model card generated by _generate_model_card()
    :param metadata: metadata
    """
    readme_path = local_path / "README.md"
    readme = ""
    if readme_path.exists():
        with readme_path.open("r", encoding="utf8") as f:
            readme = f.read()
    else:
        readme = generated_model_card

    with readme_path.open("w", encoding="utf-8") as f:
        f.write(readme)

    # Save our metrics to Readme metadata
    metadata_save(readme_path, metadata)


def _add_logdir(local_path: Path, logdir: Path):
    """Adds a logdir to the repository.
    :param local_path: repository directory
    :param logdir: logdir directory
    """
    if logdir.exists() and logdir.is_dir():
        # Add the logdir to the repository under new dir called logs
        repo_logdir = local_path / "logs"

        # Delete current logs if they exist
        if repo_logdir.exists():
            shutil.rmtree(repo_logdir)

        # Copy logdir into repo logdir
        shutil.copytree(logdir, repo_logdir)


def make_env(env_id, seed, idx, capture_video, run_name):
    def thunk():
        env = gym.make(env_id)
        env = gym.wrappers.RecordEpisodeStatistics(env)
        if capture_video:
            if idx == 0:
                env = gym.wrappers.RecordVideo(env, f"videos/{run_name}")
        env.seed(seed)
        env.action_space.seed(seed)
        env.observation_space.seed(seed)
        return env

    return thunk


def layer_init(layer, std=np.sqrt(2), bias_const=0.0):
    torch.nn.init.orthogonal_(layer.weight, std)
    torch.nn.init.constant_(layer.bias, bias_const)
    return layer


class Agent(nn.Module):
    def __init__(self, envs):
        super().__init__()
        self.critic = nn.Sequential(
```

```
            layer_init(nn.Linear(np.array(envs.single_observation_space.shape).prod(), 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, 1), std=1.0),
        )
        self.actor = nn.Sequential(
            layer_init(nn.Linear(np.array(envs.single_observation_space.shape).prod(), 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, envs.single_action_space.n), std=0.01),
        )

    def get_value(self, x):
        return self.critic(x)

    def get_action_and_value(self, x, action=None):
        logits = self.actor(x)
        probs = Categorical(logits=logits)
        if action is None:
            action = probs.sample()
        return action, probs.log_prob(action), probs.entropy(), self.critic(x)


if __name__ == "__main__":
    args = parse_args()
    run_name = f"{args.env_id}__{args.exp_name}__{args.seed}__{int(time.time())}"
    if args.track:
        import wandb

        wandb.init(
            project=args.wandb_project_name,
            entity=args.wandb_entity,
            sync_tensorboard=True,
            config=vars(args),
            name=run_name,
            monitor_gym=True,
            save_code=True,
        )
    writer = SummaryWriter(f"runs/{run_name}")
    writer.add_text(
        "hyperparameters",
        "|param|value|\n|-|-|\n%s" % ("\n".join([f"|{key}|{value}|" for key, value in vars(args).
    items()])),
    )

    # TRY NOT TO MODIFY: seeding
    random.seed(args.seed)
    np.random.seed(args.seed)
    torch.manual_seed(args.seed)
    torch.backends.cudnn.deterministic = args.torch_deterministic

    device = torch.device("cuda" if torch.cuda.is_available() and args.cuda else "cpu")

    # env setup
    envs = gym.vector.SyncVectorEnv(
        [make_env(args.env_id, args.seed + i, i, args.capture_video, run_name) for i in range(args.
    num_envs)]
    )
    assert isinstance(envs.single_action_space, gym.spaces.Discrete), "only discrete action space
    is supported"

    agent = Agent(envs).to(device)
    optimizer = optim.Adam(agent.parameters(), lr=args.learning_rate, eps=1e-5)
```

```
# ALGO Logic: Storage setup
obs = torch.zeros((args.num_steps, args.num_envs) + envs.single_observation_space.shape).to(
device)
actions = torch.zeros((args.num_steps, args.num_envs) + envs.single_action_space.shape).to(
device)
logprobs = torch.zeros((args.num_steps, args.num_envs)).to(device)
rewards = torch.zeros((args.num_steps, args.num_envs)).to(device)
dones = torch.zeros((args.num_steps, args.num_envs)).to(device)
values = torch.zeros((args.num_steps, args.num_envs)).to(device)

# TRY NOT TO MODIFY: start the game
global_step = 0
start_time = time.time()
next_obs = torch.Tensor(envs.reset()).to(device)
next_done = torch.zeros(args.num_envs).to(device)
num_updates = args.total_timesteps // args.batch_size

for update in range(1, num_updates + 1):
    # Annealing the rate if instructed to do so.
    if args.anneal_lr:
        frac = 1.0 - (update - 1.0) / num_updates
        lrnow = frac * args.learning_rate
        optimizer.param_groups[0]["lr"] = lrnow

    for step in range(0, args.num_steps):
        global_step += 1 * args.num_envs
        obs[step] = next_obs
        dones[step] = next_done

        # ALGO LOGIC: action logic
        with torch.no_grad():
            action, logprob, _, value = agent.get_action_and_value(next_obs)
            values[step] = value.flatten()
        actions[step] = action
        logprobs[step] = logprob

        # TRY NOT TO MODIFY: execute the game and log data.
        next_obs, reward, done, info = envs.step(action.cpu().numpy())
        rewards[step] = torch.tensor(reward).to(device).view(-1)
        next_obs, next_done = torch.Tensor(next_obs).to(device), torch.Tensor(done).to(device)

        for item in info:
            if "episode" in item.keys():
                print(f"global_step={global_step}, episodic_return={item['episode']['r']}")
                writer.add_scalar("charts/episodic_return", item["episode"]["r"], global_step)
                writer.add_scalar("charts/episodic_length", item["episode"]["l"], global_step)
                break

    # bootstrap value if not done
    with torch.no_grad():
        next_value = agent.get_value(next_obs).reshape(1, -1)
        if args.gae:
            advantages = torch.zeros_like(rewards).to(device)
            lastgaelam = 0
            for t in reversed(range(args.num_steps)):
                if t == args.num_steps - 1:
                    nextnonterminal = 1.0 - next_done
                    nextvalues = next_value
                else:
                    nextnonterminal = 1.0 - dones[t + 1]
                    nextvalues = values[t + 1]
                delta = rewards[t] + args.gamma * nextvalues * nextnonterminal - values[t]
                advantages[t] = lastgaelam = delta + args.gamma * args.gae_lambda *
nextnonterminal * lastgaelam
```

```
                returns = advantages + values
            else:
                returns = torch.zeros_like(rewards).to(device)
                for t in reversed(range(args.num_steps)):
                    if t == args.num_steps - 1:
                        nextnonterminal = 1.0 - next_done
                        next_return = next_value
                    else:
                        nextnonterminal = 1.0 - dones[t + 1]
                        next_return = returns[t + 1]
                    returns[t] = rewards[t] + args.gamma * nextnonterminal * next_return
                advantages = returns - values

        # flatten the batch
        b_obs = obs.reshape((-1,) + envs.single_observation_space.shape)
        b_logprobs = logprobs.reshape(-1)
        b_actions = actions.reshape((-1,) + envs.single_action_space.shape)
        b_advantages = advantages.reshape(-1)
        b_returns = returns.reshape(-1)
        b_values = values.reshape(-1)

        # Optimizing the policy and value network
        b_inds = np.arange(args.batch_size)
        clipfracs = []
        for epoch in range(args.update_epochs):
            np.random.shuffle(b_inds)
            for start in range(0, args.batch_size, args.minibatch_size):
                end = start + args.minibatch_size
                mb_inds = b_inds[start:end]

                _, newlogprob, entropy, newvalue = agent.get_action_and_value(
                    b_obs[mb_inds], b_actions.long()[mb_inds]
                )
                logratio = newlogprob - b_logprobs[mb_inds]
                ratio = logratio.exp()

                with torch.no_grad():
                    # calculate approx_kl http://joschu.net/blog/kl-approx.html
                    old_approx_kl = (-logratio).mean()
                    approx_kl = ((ratio - 1) - logratio).mean()
                    clipfracs += [((ratio - 1.0).abs() > args.clip_coef).float().mean().item()]

                mb_advantages = b_advantages[mb_inds]
                if args.norm_adv:
                    mb_advantages = (mb_advantages - mb_advantages.mean()) / (mb_advantages.std() +
 1e-8)

                # Policy loss
                pg_loss1 = -mb_advantages * ratio
                pg_loss2 = -mb_advantages * torch.clamp(ratio, 1 - args.clip_coef, 1 + args.
clip_coef)
                pg_loss = torch.max(pg_loss1, pg_loss2).mean()

                # Value loss
                newvalue = newvalue.view(-1)
                if args.clip_vloss:
                    v_loss_unclipped = (newvalue - b_returns[mb_inds]) ** 2
                    v_clipped = b_values[mb_inds] + torch.clamp(
                        newvalue - b_values[mb_inds],
                        -args.clip_coef,
                        args.clip_coef,
                    )
                    v_loss_clipped = (v_clipped - b_returns[mb_inds]) ** 2
                    v_loss_max = torch.max(v_loss_unclipped, v_loss_clipped)
                    v_loss = 0.5 * v_loss_max.mean()
```

```
                else:
                    v_loss = 0.5 * ((newvalue - b_returns[mb_inds]) ** 2).mean()

                entropy_loss = entropy.mean()
                loss = pg_loss - args.ent_coef * entropy_loss + v_loss * args.vf_coef

                optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(agent.parameters(), args.max_grad_norm)
                optimizer.step()

            if args.target_kl is not None:
                if approx_kl > args.target_kl:
                    break

        y_pred, y_true = b_values.cpu().numpy(), b_returns.cpu().numpy()
        var_y = np.var(y_true)
        explained_var = np.nan if var_y == 0 else 1 - np.var(y_true - y_pred) / var_y

        # TRY NOT TO MODIFY: record rewards for plotting purposes
        writer.add_scalar("charts/learning_rate", optimizer.param_groups[0]["lr"], global_step)
        writer.add_scalar("losses/value_loss", v_loss.item(), global_step)
        writer.add_scalar("losses/policy_loss", pg_loss.item(), global_step)
        writer.add_scalar("losses/entropy", entropy_loss.item(), global_step)
        writer.add_scalar("losses/old_approx_kl", old_approx_kl.item(), global_step)
        writer.add_scalar("losses/approx_kl", approx_kl.item(), global_step)
        writer.add_scalar("losses/clipfrac", np.mean(clipfracs), global_step)
        writer.add_scalar("losses/explained_variance", explained_var, global_step)
        print("SPS:", int(global_step / (time.time() - start_time)))
        writer.add_scalar("charts/SPS", int(global_step / (time.time() - start_time)), global_step)

    envs.close()
    writer.close()

    # Create the evaluation environment
    eval_env = gym.make(args.env_id)

    package_to_hub(
        repo_id=args.repo_id,
        model=agent,  # The model we want to save
        hyperparameters=args,
        eval_env=gym.make(args.env_id),
        logs=f"runs/{run_name}",
    )
```

Чтобы поделиться своей моделью с сообществом, нужно выполнить ещё три шага:

1. Если у вас ещё нет аккаунта Hugging Face, создайте его: https://huggingface.co/join.
2. Войдите в аккаунт и получите токен аутентификации на сайте Hugging Face.

- Создайте новый токен (https://huggingface.co/settings/tokens) **с правами записи (write)**.

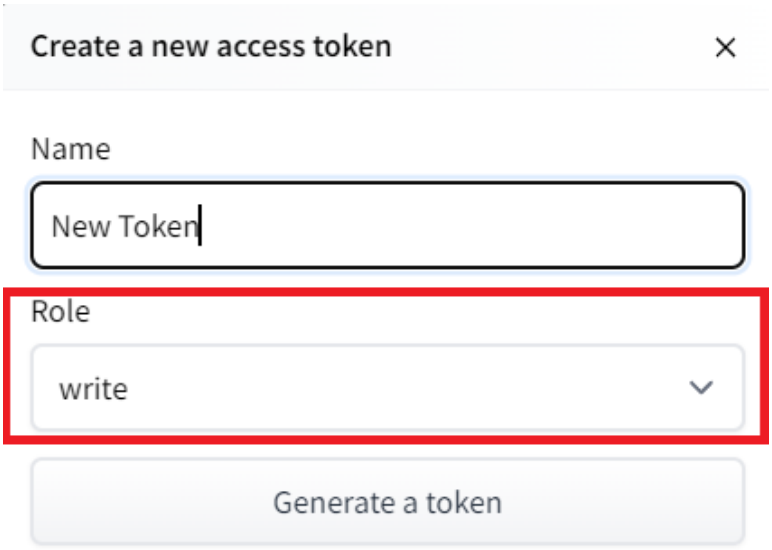


Рис. 20: Создание токена Hugging Face с правом записи [оригинал]

- Скопируйте токен.
- Выполните следующую ячейку и вставьте токен.

```
from huggingface_hub import notebook_login
notebook_login()
!git config --global credential.helper store
```

Если вы не используете Google Colab или Jupyter Notebook, вместо этого выполните команду `huggingface-cli login`.

5.9 Запуск обучения

Внимание! Внимание! Внимание! **Не используйте тот же идентификатор репозитория, который применяли в модуле 1**.

- Теперь, когда PPO реализован с нуля и добавлена интеграция с Hugging Face, можно приступить к обучению.
- Сначала скопируйте весь код в созданный вами файл `ppo.py`.

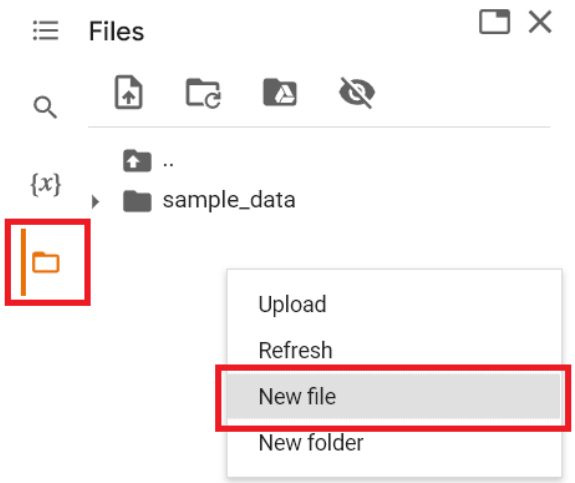


Рис. 21: Создание Python-файла с реализацией PPO [оригинал]



Рис. 22: Подготовка и сохранение скрипта PPO [оригинал]

- Запустите этот скрипт с дополнительными параметрами, заданными через `argparse`. Команда запуска: `python <name-of-python-script>.py`.
- Рекомендуется изменить и другие гиперпараметры, иначе обучение может оказаться недостаточно устойчивым.

```
!python ppo.py --env-id="LunarLander-v2" --repo-id="YOUR_REPO_ID" --total-timesteps=50000
```

5.10 Дополнительные задания

Лучший способ учиться — **самостоятельно экспериментировать**! Почему бы не попробовать другую среду? Или не модифицировать реализацию для совместимости с Gymnasium?

Увидимся во второй части модуля 8, где мы будем обучать агентов играть в Doom!

5.11 Продолжайте учиться и совершенствоваться

6 Заключение

На сегодня всё. Поздравляем с завершением этой части модуля и руководства!

Лучший способ освоить материал — практиковаться и экспериментировать. **Например, почему бы не улучшить реализацию, чтобы она принимала на вход кадры изображений?**

Увидимся во второй части модуля!

6.1 Продолжайте учиться и совершенствоваться

7 Дополнительная литература

Это **необязательные материалы** для тех, кто хочет изучить тему глубже.

7.1 Объяснения алгоритма PPO

- Даниэль Бик: последовательное самостоятельное объяснение Proximal Policy Optimization
- Как лучше понять алгоритм Proximal Policy Optimization в RL?
- Основы глубокого RL: лекция 4 об алгоритмах TRPO и PPO, Питер Аббел
- Статья OpenAI об алгоритме PPO
- Материал Spinning Up RL об алгоритме PPO
- Научная статья «Proximal Policy Optimization Algorithms»

7.2 Особенности реализации PPO

- 37 важных деталей реализации Proximal Policy Optimization
- Часть 1 из 3: реализация PPO и 11 основных деталей реализации

7.3 Выборка по значимости (importance sampling)

- Объяснение выборки по значимости (importance sampling)

8 Введение в PPO с использованием Sample Factory



Рис. 23: Модуль 8, часть 2: PPO с Sample Factory и ViZDoom [оригинал]

Во второй части модуля 8 мы глубже изучим оптимизацию PPO. С помощью Sample-Factory, **асинхронной реализации алгоритма PPO**, мы обучим агента играть в ViZDoom — открытую реализацию Doom.

В ноутбуке вы **обучите агента проходить уровень Health Gathering**, собирая аптечки, чтобы не погибнуть. После этого сможете **попробовать обучить агента в более сложных сценариях, например Deathmatch**.



Рис. 24: Игровые среды ViZDoom, используемые для обучения агентов [оригинал]

Звучит интересно? Тогда начнём!

Практическое задание подготовил Эдвард Бичинг (Edward Beeching), исследователь машинного обучения в Hugging Face. Он также работал над Godot Reinforcement Learning Agents — открытым интерфейсом для разработки сред и агентов в игровом движке Godot.

9 Практика: продвинутое глубокое RL — игра в Doom по пикселям с Sample Factory

Исходный ноутбук Google Colab

Ноутбук Colab: открыть исходный ноутбук в Colab.

9.1 Модуль 8, часть 2: игра в Doom по пикселям с Sample Factory



Рис. 25: Модуль 8, часть 2: PPO с Sample Factory и ViZDoom [оригинал]

В этом ноутбуке мы научимся обучать глубокую нейронную сеть собирать предметы в трёхмерной среде на основе игры Doom. Ниже в оригинальном курсе приведено видео поведения полученной политики. Для обучения используется Sample Factory — асинхронная реализация алгоритма PPO.

Обратите внимание на следующие моменты:

- Sample Factory — продвинутый фреймворк RL, который **работает только в Linux и macOS**, но не в Windows (согласно данному учебному материалу).
- Максимальная производительность достигается **на машине с GPU и большим числом ядер CPU**: до 100 тысяч взаимодействий со средой в секунду. Ресурсы стандартного ноутбука Colab **ограничивают производительность этой библиотеки**, поэтому скорость, полученная здесь, **не отражает её реальных возможностей**.
- Результаты измерения производительности Sample Factory в разных сценариях приведены в разделе примеров.

```
from IPython.display import HTML

HTML(
    """<video width="640" height="480" controls>
  <source src="https://huggingface.co/edbeeching/doom_health_gathering_supreme_3333/resolve/main/
    replay.mp4"
  type="video/mp4">Your browser does not support the video tag.</video>"""
)
```

Для зачёта практической части в рамках сертификации необходимо опубликовать одну модель:

- Для среды `doom_health_gathering_supreme` требуется результат не ниже $>= 5$.

Для просмотра результата найдите свою модель в таблице лидеров. **Результат вычисляется как среднее вознаграждение минус стандартное отклонение вознаграждения** (mean_reward - std).

Если вы не видите модель, **прокрутите страницу до конца и нажмите кнопку обновления**.

Подробнее о сертификации читайте в разделе https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process.

9.2 Настройка GPU

- Чтобы **ускорить обучение агента, мы используем GPU**. В Colab откройте `Runtime > Change Runtime type`.

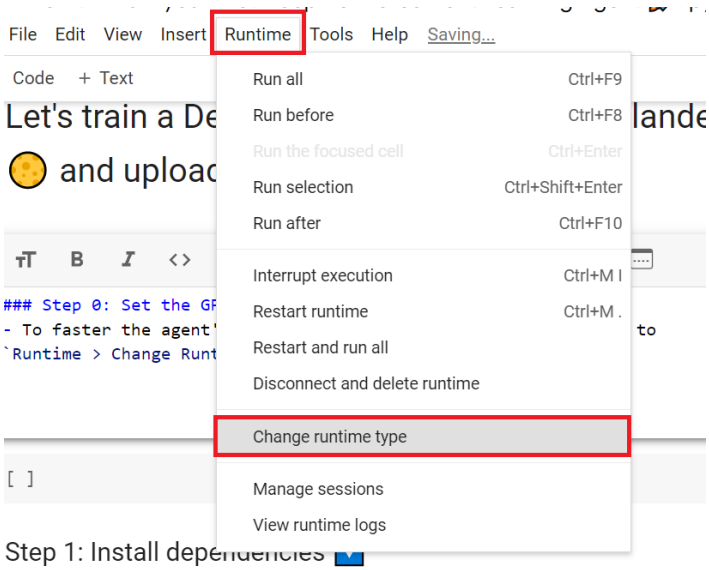


Рис. 26: Меню Google Colab для выбора среды выполнения [оригинал]

- `Hardware Accelerator > GPU`

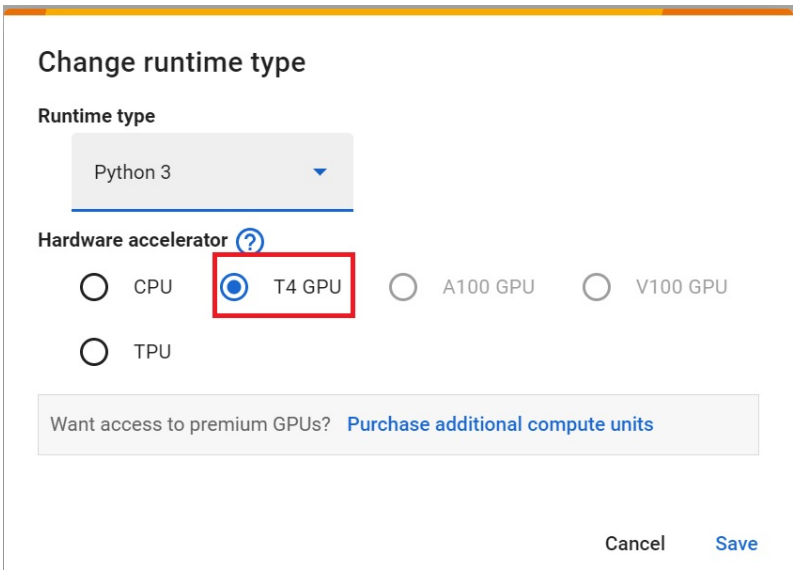


Рис. 27: Выбор GPU-ускорителя в Google Colab [оригинал]

Перед обучением агента **изучим библиотеку и среды, которые будем использовать**.

9.3 Sample Factory

Sample Factory — одна из **самых быстрых библиотек RL, ориентированная на высокоэффективные синхронные и асинхронные реализации методов градиента политики (PPO)**.

Sample Factory тщательно **протестирована, используется многими исследователями и практиками** и активно поддерживается. Наша реализация **достигает результатов уровня SOTA в различных задачах, сокращая время обучения RL-экспериментов и требования к оборудованию**.

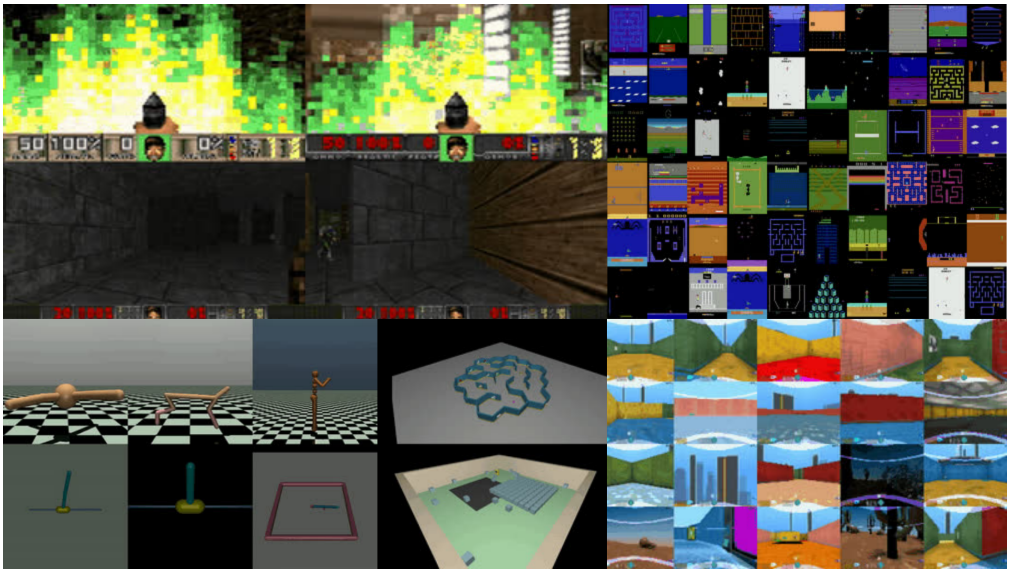


Рис. 28: Среды, поддерживаемые Sample Factory [оригинал]

Ключевые возможности

- Высокооптимизированная архитектура алгоритма для максимальной скорости сбора обучающих данных.
- Синхронный и асинхронный режимы обучения.
- Последовательный (однопроцессный) режим для удобной отладки.
- Высокая производительность как в средах на CPU, так и в средах с GPU-ускорением.
- Обучение одного или нескольких агентов, self-play, возможность одновременно обучать несколько политик на одном или нескольких GPU.
- Обучение на основе популяций (Population-Based Training, PBT).
- Дискретные, непрерывные и гибридные пространства действий.
- Пространства наблюдений на основе векторов, изображений и словарей.
- Автоматическое создание архитектуры модели по спецификациям пространств действий и наблюдений; поддерживаются пользовательские архитектуры моделей.
- Возможность встраивания в другие проекты и полноценная поддержка пользовательских сред.
- Подробные сводки WandB и TensorBoard, а также пользовательские метрики.
- Интеграция с Hugging Face для загрузки обученных моделей и метрик на Hub.
- Множество готовых интеграций со средами, включая настроенные параметры и обученные модели.

Все перечисленные политики доступны на Hub. Найдите их по тегу sample-factory.

Как работает Sample Factory

Sample Factory — одна из **наиболее оптимизированных реализаций RL, доступных сообществу**.

Она работает, **создавая несколько процессов: процессы сбора опыта (rollout workers), выполнения инференса (inference workers) и обучения (learner worker)**.

Эти *рабочие процессы* **обмениваются данными через общую память, что снижает накладные расходы на межпроцессное взаимодействие**.

Процессы сбора опыта взаимодействуют со средой и передают наблюдения *процессам инференса*.

Процессы инференса используют фиксированную версию политики для вычисления действий и **отправляют эти действия обратно процессам сбора опыта**.

После k шагов процессы сбора опыта передают траекторию взаимодействий процессу обучения, **который использует её для обновления нейросети политики агента**.

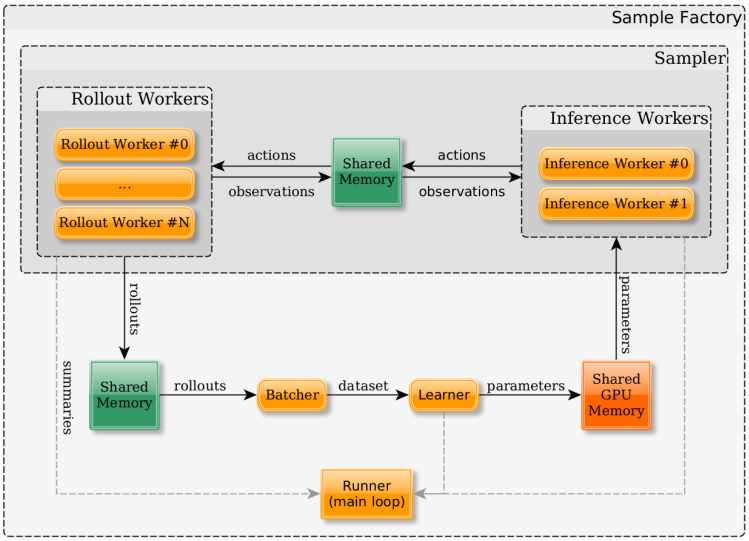


Рис. 29: Распределённая архитектура Sample Factory: сбор опыта, инференс и обучение [оригинал]

Модели Actor–Critic в Sample Factory

Модели Actor–Critic в Sample Factory состоят из трёх компонентов:

- **Encoder (кодировщик)** — обрабатывает входные наблюдения (изображения, векторы) и преобразует их в вектор признаков. Именно эту часть модели чаще всего требуется настраивать под свою задачу.
- **Core (основной блок)** — объединяет векторы от одного или нескольких кодировщиков; для агентов с памятью может включать однослойную или многослойную сеть LSTM/GRU.
- **Decoder (декодировщик)** — добавляет слои после основного блока модели перед вычислением выходов политики и ценности.

Библиотека спроектирована так, чтобы автоматически поддерживать любые пространства наблюдений и действий. Пользователи могут легко добавлять собственные модели. Подробнее об этом говорится в документации.

9.4 ViZDoom

ViZDoom — **открытый интерфейс Python к игровому движку Doom Engine**.

Библиотека создана в 2016 году Мареком Выдмухом (Marek Wydmuch) и Михалом Кемпкой (Michal Kempka) из Института вычислительных наук Познаньского технического университета (Польша).

Библиотека позволяет **обучать агентов непосредственно по пикселям экрана в разных сценариях**, в том числе в командном Deathmatch, показанном на видео ниже. Поскольку основой ViZDoom служит игра 1990-х годов, на современном оборудовании её можно запускать с

ускорением, **что даёт возможность сравнительно быстро обучать сложному поведению агентов**.

Библиотека обладает следующими возможностями:

- Кроссплатформенность (Linux, macOS, Windows).
- API для Python и C++.
- Обёртки сред для OpenAI Gym.
- Простое создание собственных сценариев (доступны визуальные редакторы, язык сценариев и примеры).
- Асинхронный и синхронный режимы одиночной и многопользовательской игры.
- Малый размер (несколько мегабайт) и высокая скорость (до 7000 кадров/с в синхронном однопоточном режиме).
- Настраиваемые разрешение и параметры рендеринга.
- Доступ к буферу глубины (трёхмерное зрение).
- Автоматическая разметка игровых объектов, видимых в кадре.
- Доступ к аудиобуферу.
- Доступ к списку игровых акторов/объектов и геометрии карты.
- Рендеринг без отображения окна и запись эпизодов.
- Масштабирование времени в асинхронном режиме.

9.5 Установка необходимых зависимостей ViZDoom

После настройки среды выполнения Colab начнём с установки зависимостей, необходимых для запуска ViZDoom в Linux.

Если вы выполняете задание локально на Mac, воспользуйтесь инструкцией по установке на странице GitHub.

```
# Install ViZDoom deps from
# https://github.com/mwydmuch/ViZDoom/blob/master/doc/Building.md#-linux

apt-get install build-essential zlib1g-dev libsdl2-dev libjpeg-dev \
nasm tar libbz2-dev libgtk2.0-dev cmake git libfluidsynth-dev libgme-dev \
libopenal-dev timidity libwildmidi-dev unzip ffmpeg

# Boost libraries
apt-get install libboost-all-dev

# Lua binding dependencies
apt-get install liblua5.1-dev
```

9.6 Установка Sample Factory и ViZDoom

- Установка может занять около 7 минут.

```
pip install sample-factory
pip install vizdoom
```

9.7 Настройка среды Doom в Sample Factory

```
import functools

from sample_factory.algo.utils.context import global_model_factory
from sample_factory.cfg.arguments import parse_full_cfg, parse_sf_args
from sample_factory.envs.env_utils import register_env
from sample_factory.train import run_rl
```

```
from sf_examples.vizdoom.doom.doom_model import make_vizdoom_encoder
from sf_examples.vizdoom.doom.doom_params import add_doom_env_args, doom_override_defaults
from sf_examples.vizdoom.doom.doom_utils import DOOM_ENVS, make_doom_env_from_spec


# Registers all the ViZDoom environments
def register_vizdoom_envs():
    for env_spec in DOOM_ENVS:
        make_env_func = functools.partial(make_doom_env_from_spec, env_spec)
        register_env(env_spec.name, make_env_func)


# Sample Factory allows the registration of a custom Neural Network architecture
# See https://github.com/alex-petrenko/sample-factory/blob/master/sf_examples/vizdoom/doom/
    doom_model.py for more details
def register_vizdoom_models():
    global_model_factory().register_encoder_factory(make_vizdoom_encoder)


def register_vizdoom_components():
    register_vizdoom_envs()
    register_vizdoom_models()


# parse the command line args and create a config
def parse_vizdoom_cfg(argv=None, evaluation=False):
    parser, _ = parse_sf_args(argv=argv, evaluation=evaluation)
    # parameters specific to Doom envs
    add_doom_env_args(parser)
    # override Doom default values for algo parameters
    doom_override_defaults(parser)
    # second parsing pass yields the final configuration
    final_cfg = parse_full_cfg(parser, argv)
    return final_cfg
```

После завершения настройки можно приступить к обучению агента. Для этого мы выбрали задачу ViZDoom под названием `Health Gathering Supreme`.

Сценарий Health Gathering Supreme



Рис. 30: Сценарий ViZDoom «Health Gathering Supreme» [оригинал]

Цель сценария — **научить агента выживать, не сообщая ему напрямую, что именно**

обеспечивает выживание. Агент знает лишь то, что **жизнь ценна**, а смерть нежелательна. Поэтому **ему предстоит самостоятельно выяснить, что продлевает его существование и как здоровье связано с выживанием**.

Карта представляет собой прямоугольник со стенами и зелёным кислотным полом, который **периодически наносит урон игроку**. Изначально аптечки равномерно распределены по карте; время от времени с неба падает новая аптечка. **Аптечки восстанавливают часть здоровья игрока**; чтобы выжить, агент должен подбирать их. Эпизод заканчивается после смерти игрока либо по истечении отведённого времени.

Дополнительные параметры сценария:

- Living_reward = 1.
- Три доступных действия: повернуть налево, повернуть направо, двигаться вперёд.
- Одна доступная игровая переменная: HEALTH.
- Штраф за смерть = 100.

Подробнее о доступных сценариях ViZDoom можно прочитать здесь.

Существуют и более сложные сценарии ViZDoom, например представленные на этой странице GitHub.

9.8 Обучение агента

- Обучим агента в течение 4 000 000 шагов. Это займёт приблизительно 20 минут.

```
## Start the training, this should take around 15 minutes
register_vizdoom_components()

# The scenario we train on today is health gathering
# other scenarios include "doom_basic", "doom_two_colors_easy", "doom_dm", "doom_dwango5", "
    doom_my_way_home", "doom_deadly_corridor", "doom_defend_the_center", "doom_defend_the_line"
env = "doom_health_gathering_supreme"
cfg = parse_vizdoom_cfg(
    argv=[f"--env={env}", "--num_workers=8", "--num_envs_per_worker=4", "--train_for_env_steps
    =4000000"]
)

status = run_rl(cfg)
```

9.9 Оценка обученной политики и запись видео

```
from sample_factory.enjoy import enjoy

cfg = parse_vizdoom_cfg(
    argv=[f"--env={env}", "--num_workers=1", "--save_video", "--no_render", "--max_num_episodes
    =10"], evaluation=True
)
status = enjoy(cfg)
```

9.10 Визуализация поведения агента

```
from base64 import b64encode
from IPython.display import HTML

mp4 = open("/content/train_dir/default_experiment/replay.mp4", "rb").read()
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
```

```
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

Агент кое-чему научился, но его результаты могли бы быть лучше. Очевидно, стоило бы продолжить обучение. Однако сначала опубликуем эту модель на Hub.

9.11 Публикация контрольной точки и видео на Hugging Face Hub

Чтобы поделиться моделью с сообществом, необходимо выполнить ещё три шага:

1. Если аккаунта Hugging Face ещё нет, создайте его: https://huggingface.co/join.
2. Войдите в аккаунт и получите токен аутентификации на сайте Hugging Face.

- Создайте новый токен (https://huggingface.co/settings/tokens) **с правами записи (write)**.

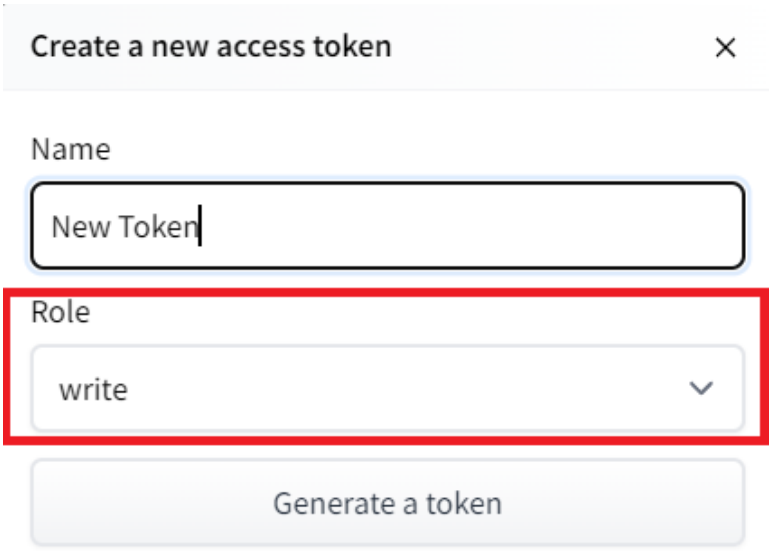


Рис. 31: Создание токена Hugging Face с правом записи [оригинал]

- Скопируйте токен.
- Выполните следующую ячейку и вставьте токен.

Если вы не используете Google Colab или Jupyter Notebook, вместо этого выполните команду `huggingface-cli login`.

```
from huggingface_hub import notebook_login
notebook_login()
!git config --global credential.helper store
```

```
from sample_factory.enjoy import enjoy

hf_username = "ThomasSimonini"  # insert your HuggingFace username here

cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
```

```
        "--no_render",
        "--max_num_episodes=10",
        "--max_num_frames=100000",
        "--push_to_hub",
        f"--hf_repository={hf_username}/rl_course_vizdoom_health_gathering_supreme",
    ],
    evaluation=True,
)
status = enjoy(cfg)
```

9.12 Загрузка другой модели

Качество этой модели уже неплохое, но можно добиться большего! Загрузим с Hub агента, которого обучали в течение 10 миллиардов шагов, и посмотрим на его поведение.

```
#download the agent from the hub
python -m sample_factory.huggingface.load_from_hub -r edbeeching/doom_health_gathering_supreme_2222
     -d ./train_dir
```

```
ls train_dir/doom_health_gathering_supreme_2222
```

```
env = "doom_health_gathering_supreme"
cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
        "--no_render",
        "--max_num_episodes=10",
        "--experiment=doom_health_gathering_supreme_2222",
        "--train_dir=train_dir",
    ],
    evaluation=True,
)
status = enjoy(cfg)
```

```
mp4 = open("/content/train_dir/doom_health_gathering_supreme_2222/replay.mp4", "rb").read()
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

9.13 Дополнительные задания: Doom Deathmatch

Обучение агента для Doom Deathmatch **занимает много часов и требует более мощной машины, чем доступная в Colab**.

К счастью, **мы уже обучили агента в этом сценарии и опубликовали его на Hub!** Загрузим модель и посмотрим, как он играет.

```
# Download the agent from the hub
python -m sample_factory.huggingface.load_from_hub -r edbeeching/doom_deathmatch_bots_2222 -d ./
```

```
    train_dir
```

Поскольку агент играет достаточно долго, создание видео может занять **10 минут**.

```
from sample_factory.enjoy import enjoy

register_vizdoom_components()
env = "doom_deathmatch_bots"
cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
        "--no_render",
        "--max_num_episodes=1",
        "--experiment=doom_deathmatch_bots_2222",
        "--train_dir=train_dir",
    ],
    evaluation=True,
)
status = enjoy(cfg)
mp4 = open("/content/train_dir/doom_deathmatch_bots_2222/replay.mp4", "rb").read()
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

Вы **можете попробовать обучить собственного агента в этой среде** с использованием кода выше, но не в Colab. **Удачи!**

Если хочется более простой сценарий, **попробуйте обучить агента в другой среде ViZDoom, например** `doom_deadly_corridor` **или** `doom_defend_the_center`.

Это был последний основной модуль курса. Но мы ещё не закончили! В следующем **бонусном разделе рассматриваются некоторые из наиболее интересных, продвинутых и современных направлений глубокого обучения с подкреплением**.

9.14 Продолжайте учиться и совершенствоваться

10 Заключение

На сегодня всё. Поздравляем с завершением модуля и практического руководства!

Теперь, когда вы успешно обучили агента для Doom, почему бы не попробовать Deathmatch? Помните, что этот уровень значительно сложнее только что пройденного, **но это интересный эксперимент, который мы рекомендуем провести**.

Если попробуете, обязательно поделитесь своей моделью в канале `#rl-i-made-this` на нашем Discord-сервере.

Это последний основной модуль, однако курс ещё не закончен! В следующем **бонусном модуле мы рассмотрим некоторые из самых интересных, передовых и актуальных исследований в области глубокого обучения с подкреплением**.

До следующей встречи!

10.1 Продолжайте учиться и совершенствоваться